

**CEBU INSTITUTE OF TECHNOLOGY - UNIVERSITY
COLLEGE OF COMPUTER STUDIES**

Software Requirements Specification

For

**RigMD: A Symptom-Guided Diagnostic and Decision Support System
for Non-Technical Windows Desktop PC Users**

Team Code: 2526-sem2-it332-11

Semester 2, AY 2025-2026

Prepared by:

Libato, Michael Grant

Macansantos, Axcelson

Ruperez, Raymart

Maestrado, Ralph Keane

Labaya, Godwin

Change History

Version	Date	Description	Author
1.0	May 2026	Initial draft of the Software Requirements Specification.	Michael Grant S. Libato
1.1	May 22, 2026		Ralph Keane A. Maestrado

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions, Acronyms, and Abbreviations

1.4 References

2. Overall Description

2.1 Product Perspective

2.2 User Characteristics

2.3 Product Functions

2.4 Constraints

2.5 Assumptions and Dependencies

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 Hardware Interfaces

3.1.2 Software Interfaces

3.1.3 Communications Interfaces

3.2 Functional Requirements and Wireframes

Module 1: System Profile and System Overview Module

Module 2: New Diagnosis and Automatic Scanning Module

Module 3: Evidence-Based Diagnostic Interpretation Engine Module

Module 4: Advisory Action Module

Module 5: Transparent Result Dashboard and Report Output Module

Module 6: Session History and Recurring Pattern Detection Module

3.3 Data Requirements

3.4 Non-Functional Requirements

4. Validation and Evaluation Requirements

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification is to define the functional, non-functional, interface, data, and evaluation requirements for RigMD: A Symptom-Guided Diagnostic and Decision Support System for Non-Technical Windows Desktop PC Users. The document serves as the technical reference for the development team, academic evaluators, and project stakeholders during the design, development, testing, and evaluation of the prototype for Semester 2, AY 2025-2026.

RigMD is specified as a decision-support desktop application. It does not claim to perform confirmed physical diagnosis or replace qualified hardware inspection. Instead, it helps users interpret Windows-observable PC symptoms, relate those symptoms to their system profile, and decide whether to monitor, maintain, troubleshoot, or escalate the issue for professional inspection.

1.2 Scope

RigMD is a Windows-based diagnostic and decision-support application intended for individual consumer desktop PC owners who experience Windows-observable performance, stability, or system-related symptoms. The system provides a session-based diagnostic workflow in which users initiate a Full, Component, or Scenario-based scan, while the Windows Agent collects fresh system telemetry and diagnostic evidence. Where appropriate, users may also provide structured symptom information to supplement the collected evidence. The system analyzes the available system profile, current telemetry, diagnostic evidence, and relevant symptom context to generate an evidence-based advisory result. The result dashboard presents the probable cause, recommended action, supporting evidence, and relevant next steps, while the completed diagnostic session is retained according to the application's configured data storage and access architecture.

The system targets symptoms that can be observed through the Windows environment, such as slow boot, freezing, repeated blue screen errors, driver conflict notifications, S.M.A.R.T. storage health alerts, repeated application crashes, no-display behavior after driver changes, and other OS-level performance or stability indicators.

The system includes the following six core modules:

- System Profile Module - stores or confirms key desktop PC specifications used as diagnostic context, including CPU model, RAM capacity, storage type and capacity, operating system version, GPU and chipset driver versions, and system age.
- New Diagnosis and Automatic Scanning Module - allows users to initiate Full, Component, or Scenario-based diagnostic scans. The Windows Agent automatically collects fresh system telemetry and diagnostic evidence relevant to the selected scan. Where appropriate, structured symptom information may be provided to supplement the collected evidence. .
- Evidence-Based Diagnostic Interpretation Engine Module- evaluates current system telemetry, system profile information, diagnostic evidence, relevant symptom context, and applicable prior session information to identify and rank probable internal cause categories.
- Advisory Action Module - classifies diagnostic findings into one of four action categories: Monitor, Maintain, Troubleshoot, or Escalate for Professional Inspection, and manages supported controlled-remediation workflows when an eligible corrective action is available.
- Transparent Result Dashboard - presents the advisory result, probable internal cause, confidence label, plain-language explanation, system profile details used, warning signs reference, and recommended next steps.
- Session History and Recurring Pattern Detection Module - stores diagnostic sessions locally and flags repeated symptoms or internal cause categories across two or more sessions.

RigMD is scoped to Windows desktop PCs that are functional or partially functional and can provide observable Windows-level symptoms. Laptops, mobile devices, servers, workstations, and enterprise-managed environments are outside the project scope. Fully non-functional units are limited to a safety-oriented boot-status triage path that recommends professional inspection when the condition exceeds software-layer resolution.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification.
RigMD	The proposed Windows desktop diagnostic decision-support application.
PC	Personal Computer; in this document, it refers to a consumer Windows desktop PC.
AI	Artificial Intelligence.
CPU	Central Processing Unit.

GPU	Graphics Processing Unit.
RAM	Random Access Memory.
PSU	Power Supply Unit.
SSD	Solid-State Drive.
HDD	Hard Disk Drive.
S.M.A.R.T.	Self-Monitoring, Analysis, and Reporting Technology; storage health monitoring data used as an observable warning indicator.
WMI	Windows Management Instrumentation; a Windows system information interface that may be used to assist local system profile detection.
SupaBase	A backend platform used by RigMD to facilitate communication and coordination between the ASP.NET Core backend and the Windows Agent, including the exchange of system snapshots, diagnostic evidence, predefined commands, and command results where applicable.
Gemini	The AI model API used to generate constrained plain-language explanations for diagnostic results.
SUS	System Usability Scale; a standardized usability evaluation instrument.
MVP	Minimum Viable Product.
NFR	Non-Functional Requirement.
UI	User Interface.
OS	Operating System.
AY	Academic Year.
Windows-observable symptom	A PC issue visible through Windows behavior, such as boot delay, freezing, crash reports, driver warnings, event log errors, S.M.A.R.T. alerts, or repeated application failures.
Diagnostic Session	A single guided workflow in which the user answers structured questions and receives an advisory output.

System Profile	A stored or confirmed record of the user's desktop PC specifications used as context for diagnostic interpretation.
Internal Cause Category	A probable source category identified by the diagnostic engine, such as OS performance degradation, driver conflict, storage health behavior, or boot/startup failure.
Advisory Output	The system-generated result containing the probable internal cause, action category, explanation, confidence label, warning signs, and recommended next steps.
Action Category	The practical next-step classification assigned to a result: Monitor, Maintain, Troubleshoot, or Escalate for Professional Inspection.
Recurring Pattern	A repeated symptom type or internal cause category detected across two or more sessions for the same system profile.
Decision Confidence	The user's self-reported confidence in understanding the PC problem, likely cause, recommendation, and next action.

1.4 References

- Dahouk, A. W., & Abu-Naser, S. S. (2018). A proposed knowledge based system for desktop PC troubleshooting. *International Journal of Academic Pedagogical Research*, 2(6), 1–8.
- Ravasio, P., Schär, S. G., & Krueger, H. (2004). In pursuit of desktop evolution: User problems and practices with modern desktop systems. *ACM Transactions on Computer-Human Interaction*, 11(2), 156–180.
- Kostopoulos, G., Davrazos, G., & Kotsiantis, S. (2024). Explainable artificial intelligence-based decision support systems: A recent review. *Electronics*, 13(14), 2842.
- Ravi, M., Negi, A., Bommi, N. S., & Rouf, N. (2025). Evolution of AI-driven decision making with decision support systems, expert systems, recommender systems, and XAI. *IETE Technical Review*, 42(4), 428-465.
- Zhang, W., Yang, D., & Wang, H. (2019). Data-driven methods for predictive maintenance of industrial equipment: A survey. *IEEE Systems Journal*, 13(3), 2213–2227.

Brooke, J. (1996). SUS: A quick and dirty usability scale. In *Usability Evaluation in Industry* (pp. 189–194). Taylor and Francis.

Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann.

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.

2. Overall Description

2.1 Product Perspective

RigMD is an independent Windows-based diagnostic and decision-support application designed for individual consumer desktop PC users. It is not an extension of an existing diagnostic utility and is intended to consolidate diagnostic, monitoring, advisory, and remediation functions that are commonly distributed across separate Windows troubleshooting tools, hardware monitoring utilities, driver diagnostic tools, and general-purpose AI services. RigMD is designed to provide a brand-agnostic and accessible diagnostic experience for non-technical users.

The system follows a structured diagnostic workflow rather than an open-ended chatbot interaction. Users can initiate a new diagnosis through Full, Component, or Scenario-based scanning, with the Windows Agent collecting fresh system telemetry and diagnostic evidence relevant to the selected scan. Where appropriate, structured symptom information may be provided by the user to supplement the collected system evidence. This approach allows diagnostic sessions to remain consistent, testable, and comparable against expert-validated scenarios while reducing the need for users to interpret technical system information themselves.

The prototype will be implemented using a React/Vite frontend and an ASP.NET Core backend, supported by a dedicated Windows Agent responsible for system-level telemetry collection, system snapshot generation, evidence gathering, and approved system-level operations. Supabase will facilitate communication and coordination between the backend and Windows Agent. The Diagnostic Engine will perform automatic, evidence-based analysis using current system telemetry, system profile information, diagnostic evidence, and relevant symptom context to identify probable internal causes and determine appropriate advisory actions. Controlled remediation mechanisms allow approved corrective actions to be issued to and executed by the Windows Agent. Before execution, applicable safety checks must be completed and the user must provide consent where required. Supported remediation actions may include a dry-run or preview before the actual change is applied. The Windows Agent validates each remediation request against a predefined set of

supported actions and returns the execution status and available verification evidence to the application. RigMD does not permit arbitrary CMD, PowerShell, shell, script, or unrestricted command-string execution.

2.2 User Characteristics

The primary users are individual Windows desktop PC owners who lack the technical background to interpret PC symptoms or raw monitoring data independently. Target users include:

- Students who rely on desktop PCs for academic work and may not know how to diagnose performance problems.
- Gamers who experience slow boot, freezing, stuttering, crashes, driver warnings, or performance degradation.
- Beginner PC builders who need guided support after assembling or configuring a desktop PC.
- Budget desktop users who want to avoid unnecessary service costs or incorrect troubleshooting steps.
- Content creators who depend on desktop PC performance for video editing, rendering, streaming, or heavy application use.
- General home users who need plain-language guidance when their desktop PC becomes unstable or slow.

Users are expected to have basic computer literacy, such as opening a Windows desktop application, selecting options, reading simple explanations, and following guided prompts. They are not expected to understand hardware diagnostics, event logs, S.M.A.R.T. status, or driver conflict analysis without assistance.

2.3 Product Functions

- Collect fresh system telemetry and diagnostic evidence through the Windows Agent.
- Generate system snapshots containing relevant current system information for diagnostic analysis.
- Allow the user to initiate Full, Component, or Scenario-based diagnostic scans.
- Collect structured symptom information when additional user context is required or useful.
- Analyze collected evidence using deterministic, evidence-based diagnostic logic.
- Identify and rank probable internal cause categories based on available evidence.
- Generate a probable advisory finding rather than claiming a confirmed physical diagnosis.

- Classify the recommended next action into Monitor, Maintain, Troubleshoot, or Escalate for Professional Inspection.
- Present the result through a transparent dashboard with supporting evidence, reasoning factors, confidence information, warning signs, and recommended next steps.
- Maintain diagnostic session history according to the application's current storage and backend architecture.
- Detect recurring symptoms, evidence patterns, or internal cause categories across diagnostic sessions.
- Generate technician-ready diagnostic reports.
- Provide controlled remediation for supported issues when an eligible action is available.
- Perform applicable safety checks and dry-run evaluation before supported remediation actions.
- Obtain required user consent before executing applicable remediation actions.
- Send only predefined remediation actions to the Windows Agent.
- Return remediation execution status and available verification results to the application.
- Prevent arbitrary CMD, PowerShell, shell, script, or unrestricted command execution.
- Continue deterministic diagnosis when optional AI-assisted reasoning or explanation functionality is unavailable.

2.4 Constraints

Platform Constraint: The prototype is limited to Windows desktop operating systems. macOS, Linux, mobile devices, laptops, servers, workstations, and enterprise-managed systems are outside the current scope.

Diagnostic Scope Constraint: The full diagnostic workflow applies only to functional or partially functional desktop PCs with Windows-observable symptoms. Fully non-functional units are handled only through a safety-oriented boot-status triage checklist.

System Profile Constraint: The system profile relies on local system information where technically feasible. The current wireframe scope presents system profile data as automatically detected or confirmed by the application; incomplete or low-confidence detection must be flagged clearly to the user.

Decision-Support Constraint: The system produces probable advisory findings, not guaranteed diagnoses. It cannot perform physical inspection, voltage measurement, component substitution, or electrical testing.

No Replacement or Upgrade Recommendation Constraint: The system does not directly recommend component replacement or upgrade purchases. It recommends monitoring, maintenance, software-layer troubleshooting, or professional inspection when symptoms exceed the supported advisory scope.

Data Storage and Access Constraint: User profile data, diagnostic session information, and system evidence may be stored through the application's backend and Supabase services to support Agent communication, system snapshots, diagnostic workflows, and controlled remediation. Data collection and retention must be limited to information required for the prototype's intended functions. User account management and unrestricted remote access to user data are outside the scope of the prototype.

AI Scope Constraint: AI-assisted reasoning or explanation functionality is constrained by structured diagnostic context, system profile information, collected evidence, and deterministic diagnostic output. The AI component does not operate as an unconstrained open-ended prompt interface and shall not override diagnostic safety boundaries, remediation restrictions, or predefined system rules.

Connectivity Constraint: Network connectivity is required for functions that depend on communication between the application backend, Supabase services, and the Windows Agent, as well as for optional external AI-assisted functionality. Core deterministic diagnostic logic shall remain available without depending exclusively on an external AI service.

Time Constraint: The system must reach prototype-level completion by the end of Semester 2, AY 2025-2026.

Evaluation Constraint: Expert validation requires at least two hardware-knowledgeable evaluators and at least 20 controlled diagnostic scenarios. Usability evaluation requires at least 10 target users.

2.5 Assumptions and Dependencies

- The Windows Agent has the permissions necessary to collect the system information and diagnostic evidence required by supported diagnostic functions.
- System information availability may vary depending on Windows permissions, device configuration, installed drivers, hardware interfaces, and operating system behavior.
- Users provide accurate symptom context when the system requests additional information.
- Diagnostic results depend on the quality, completeness, and availability of collected system evidence.

- AI-assisted reasoning or explanation functionality depends on the availability of the configured AI service and network connectivity.
- If the AI-assisted service is unavailable, deterministic diagnostic logic and fallback explanation mechanisms remain available.
- The Windows Agent can communicate with the ASP.NET Core backend through the configured Supabase communication mechanism when connectivity is available.
- Controlled remediation depends on the availability of a supported remediation action, successful safety checks, appropriate authorization, and successful Agent communication.
- The Windows Agent validates all remediation instructions before execution.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 Hardware Interfaces

The system interacts with the following hardware and local device resources:

- **Local System Resources:** The Windows Agent accesses supported Windows system interfaces and local resources required for telemetry collection and diagnostic evidence gathering.
- **System Information Interfaces:** The Windows Agent may query WMI, Windows APIs, event information, storage-health interfaces, driver information, and other supported system interfaces to collect CPU, RAM, storage, operating system, GPU, driver, and related system information.
- **Display:** Requires a minimum recommended display resolution of 1280 x 720 pixels to render the desktop interface correctly.
- **Input Devices:** Uses standard keyboard and mouse input for navigation, answer selection, search, filters, and export commands.
- **Storage:** Requires local disk space for the installed application, SQLite database, generated reports, and cached session records. Estimated prototype footprint should remain below 500 MB excluding user-exported reports.

3.1.2 Software Interfaces

- **Operating System:** Windows 10 (version 1903 or later) and Windows 11.
- **Application Framework:** React with Vite for the web-based user interface and client-side application components.

- **Backend Framework:** ASP.NET Core for backend services, API endpoints, diagnostic processing, and coordination between system components.
- **Windows Agent:** A dedicated Windows Agent for system information collection, diagnostic evidence gathering, backend communication, and execution of approved system-level operations.
- **Diagnostic Engine:** An automatic, evidence-based diagnostic mechanism that evaluates structured symptom responses and collected system evidence to identify and rank potential causes.
- **Agent Communication:** Supabase for communication and coordination between the ASP.NET Core backend and the Windows Agent.
- **AI-Assisted Reasoning or Explanation Service:** An optional provider-neutral AI service may be used to assist with structured diagnostic reasoning or plain-language explanation. The service shall operate using constrained diagnostic context and shall not function as an unrestricted diagnostic or command-execution interface.
- **Report Output Support:** Local export functionality for technician-ready diagnostic summaries, including copy, print, and PDF export actions as represented in the Reports wireframe.

3.1.3 Communications Interfaces

RigMD requires network communication to support coordination between the ASP.NET Core backend and the Windows Agent through Supabase. The Windows Agent is responsible for collecting system information and diagnostic evidence, submitting system snapshots, receiving approved diagnostic or remediation commands, and returning command results to the backend. The frontend communicates with the backend through defined API endpoints for diagnostic sessions, evidence, results, and controlled remediation workflows. AI-assisted functionality, where applicable, may also require secure communication with the configured AI service.

- **Backend and Agent Communication:** Communication between the ASP.NET Core backend and the Windows Agent must use Supabase services and appropriate secure communication mechanisms for transmitting system snapshots, diagnostic evidence, commands, and command results.
- **Frontend and Backend Communication:** The React/Vite frontend must communicate with the ASP.NET Core backend through defined API endpoints for submitting user inputs, retrieving diagnostic results, viewing evidence, and initiating approved workflows.
- **AI API Communication:** Any communication with an external AI service must use HTTPS and must transmit only the structured diagnostic context required for the requested operation.
- **Data Minimization:** The system must not transmit personally identifiable information, unnecessary local files, or complete session histories to external AI services. System snapshots

and diagnostic evidence transmitted between the Agent and backend must be limited to information required for diagnosis and approved system operations.

- **Agent Command Handling:** Commands sent to the Windows Agent must be explicitly defined and controlled. The Agent must validate received commands and return the corresponding execution status and results to the backend.
- **Connectivity Handling:** If communication with the backend, Supabase, or an external AI service is unavailable, the system must provide an appropriate user-facing notification and handle the affected operation without presenting incomplete or unverified results as successful.
- **Data Storage and Synchronization:** The prototype may use remote services to support Agent communication, system snapshot handling, and application workflows. Storage and synchronization requirements must therefore account for the current client-server and Agent architecture rather than assuming that all application data and major functions are exclusively local or offline.

3.2 Functional Requirements and Wireframes

The following functional requirements are organized according to the six core modules described in the proposal and the wireframes prepared for the SRS. Each module includes requirements and the corresponding wireframe reference.

Module 1: System Profile and System Overview Module

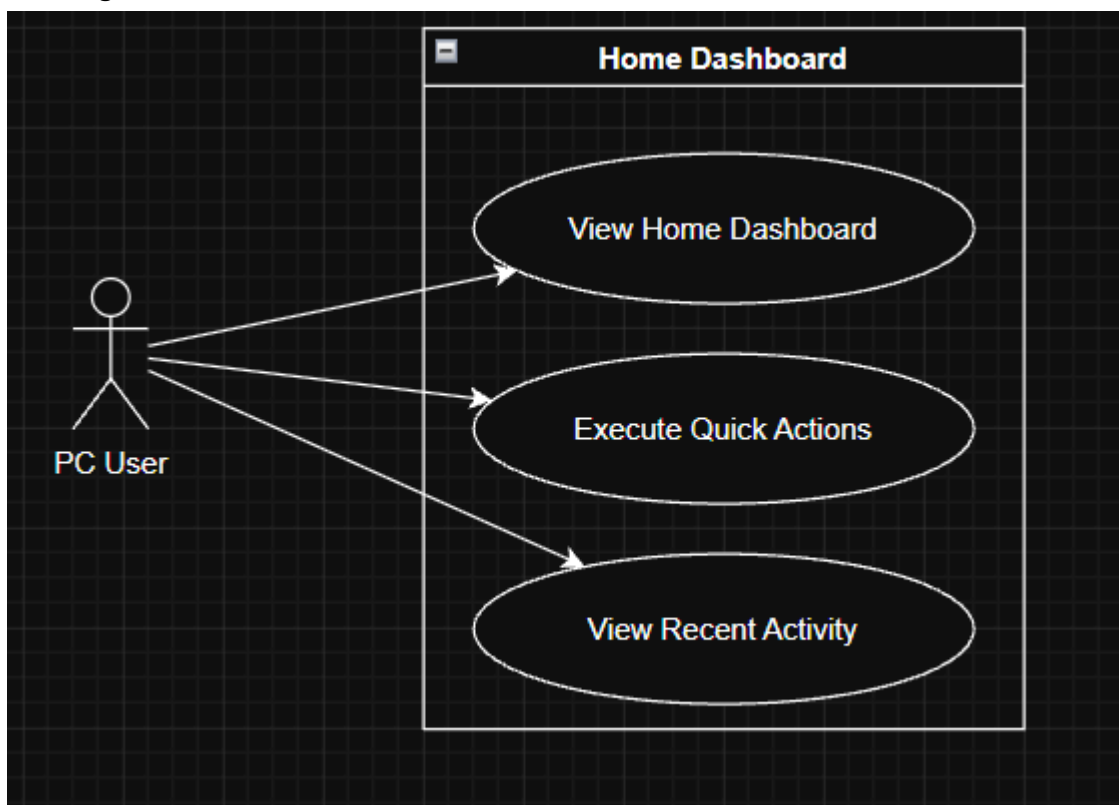
1.1 Home Dashboard & Quick Actions

Use Case Description:

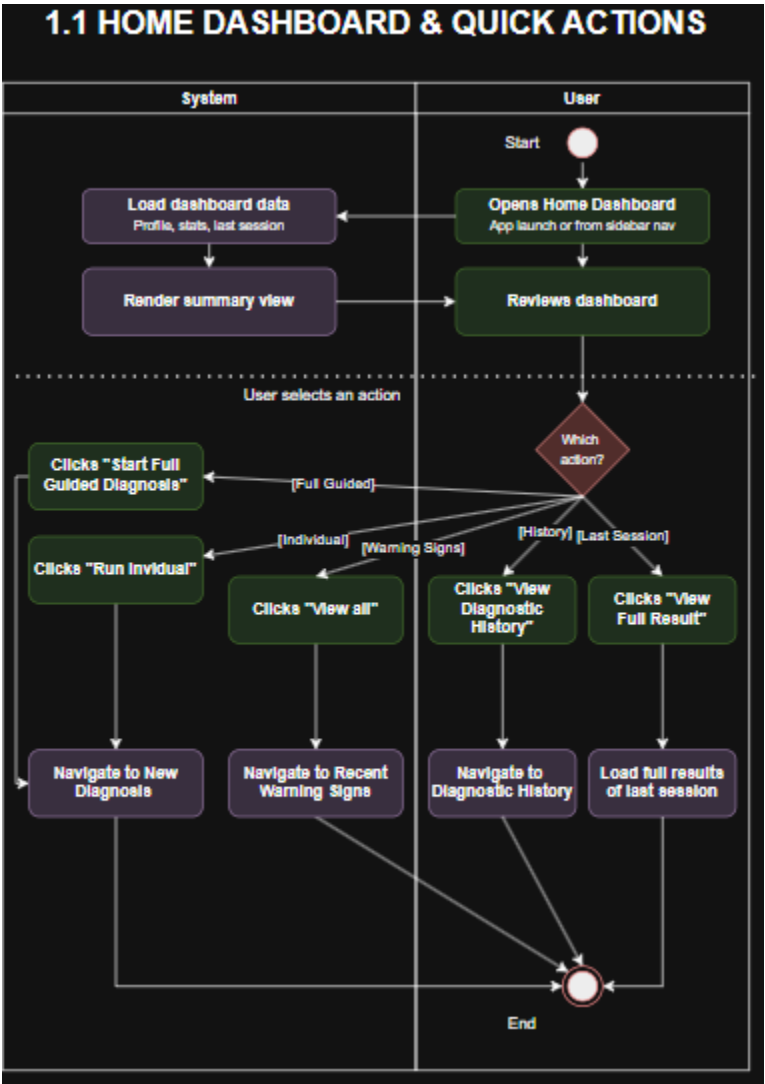
Use Case ID	UC-1.1
Use Case Name	Home Dashboard & Quick Actions
Actor	PC User

Precondition	The application is launched, and the main interface loads.
Main Flow	The user navigates to the Home Dashboard. The system displays a high-level summary of the system's status, recent diagnostic activity trends, and active warning signs. The user executes quick action shortcuts to instantly start a new diagnostic scan or review their history without navigating through multiple menus.
Alternative Flow	If no recent activity exists, the system displays a welcome prompt to start a new scan.
Postcondition	The user selects a quick action or navigates to another module.
Non-functional Requirement	The application shall launch and display the main dashboard within 5 seconds on a device meeting the minimum requirements.

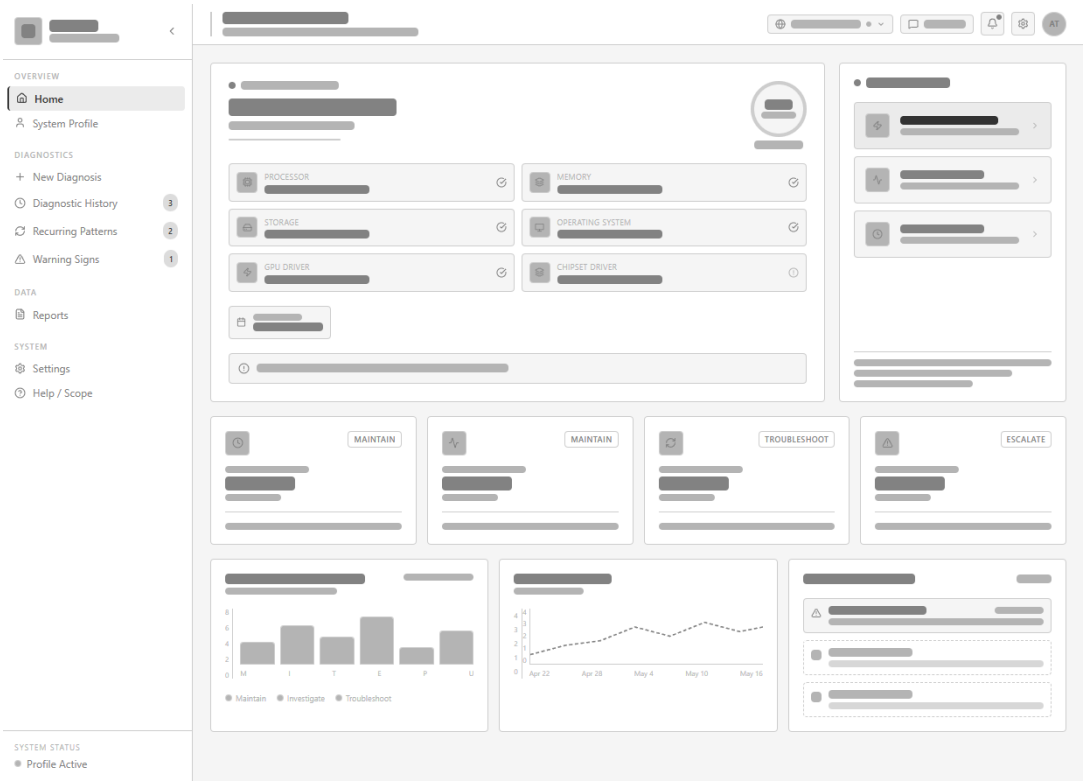
Use Case Diagram



Activity Diagram



Wireframe



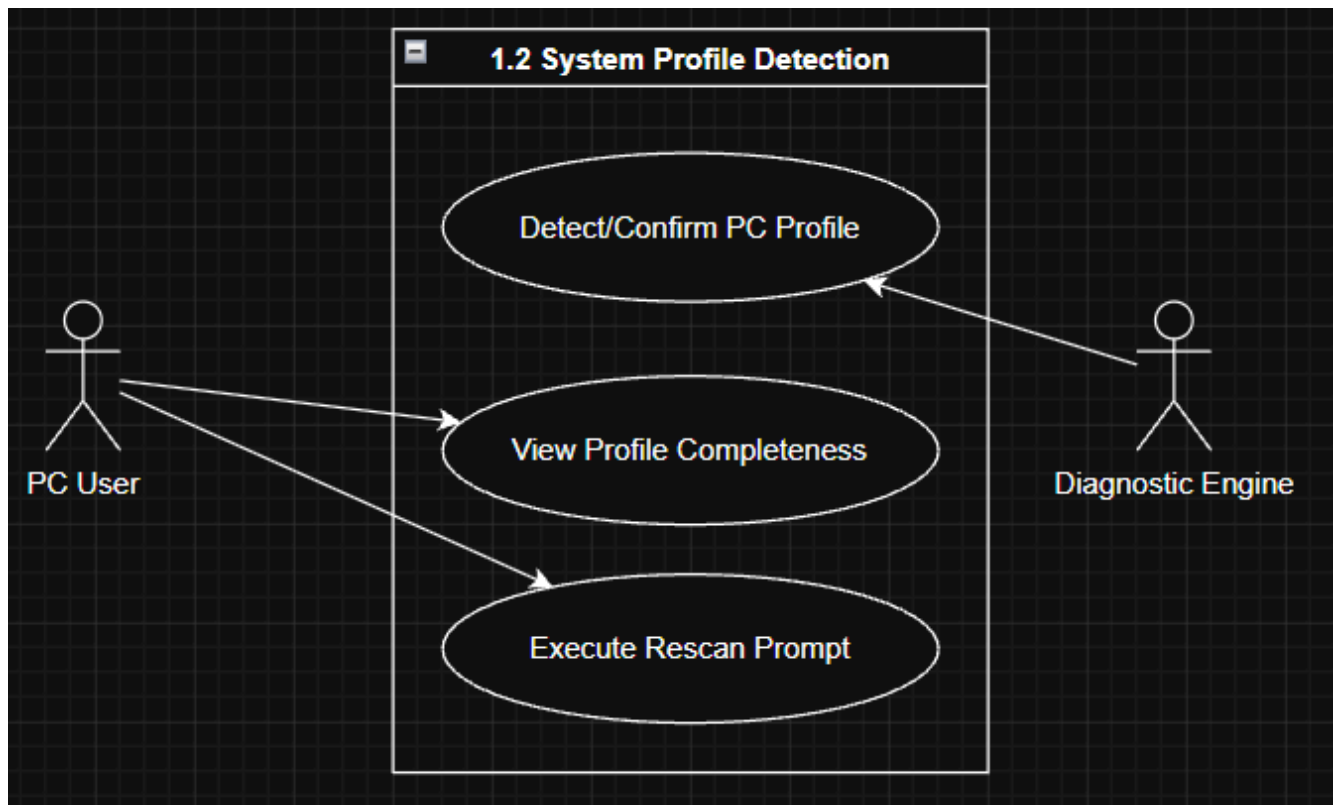
1.2 System Profile Detection

Use Case Description

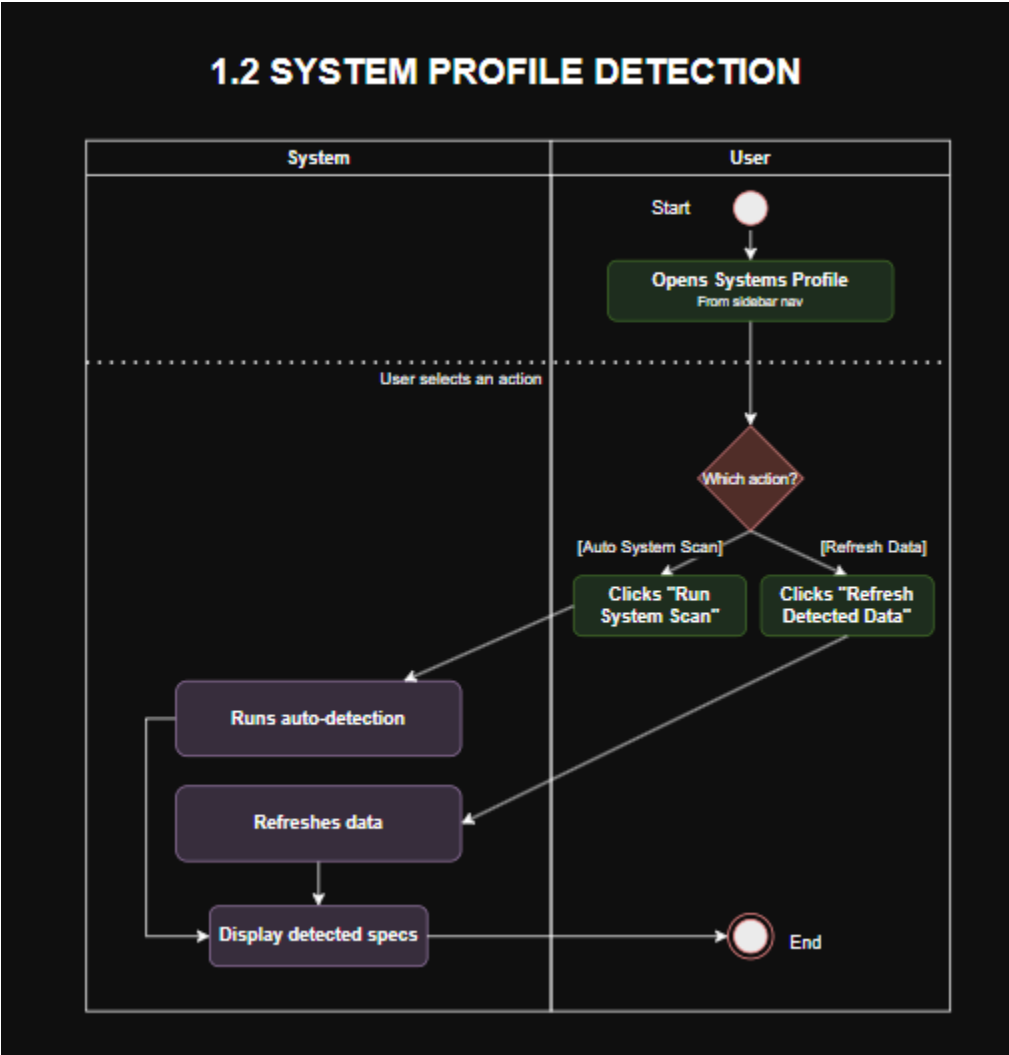
Use Case ID	UC-1.2
Use Case Name	System Profile Detection
Actor	Diagnostic Engine, PC User
Precondition	The application is launched.

Main Flow	The Diagnostic Engine automatically scans and detects the hardware and software specifications of the desktop PC. The user views this profile to verify the completeness and confidence level of the detection. The user executes a rescan prompt if any system data is missing or displays a low-confidence warning since manual editing is disabled.
Alternative Flow	If the system cannot detect certain hardware due to permissions, it flags the missing data and prompts the user to check Windows settings.
Postcondition	The system profile is saved locally and used as context for diagnostics.
Non-functional Requirement	All system profile records shall be stored locally on the user's device.

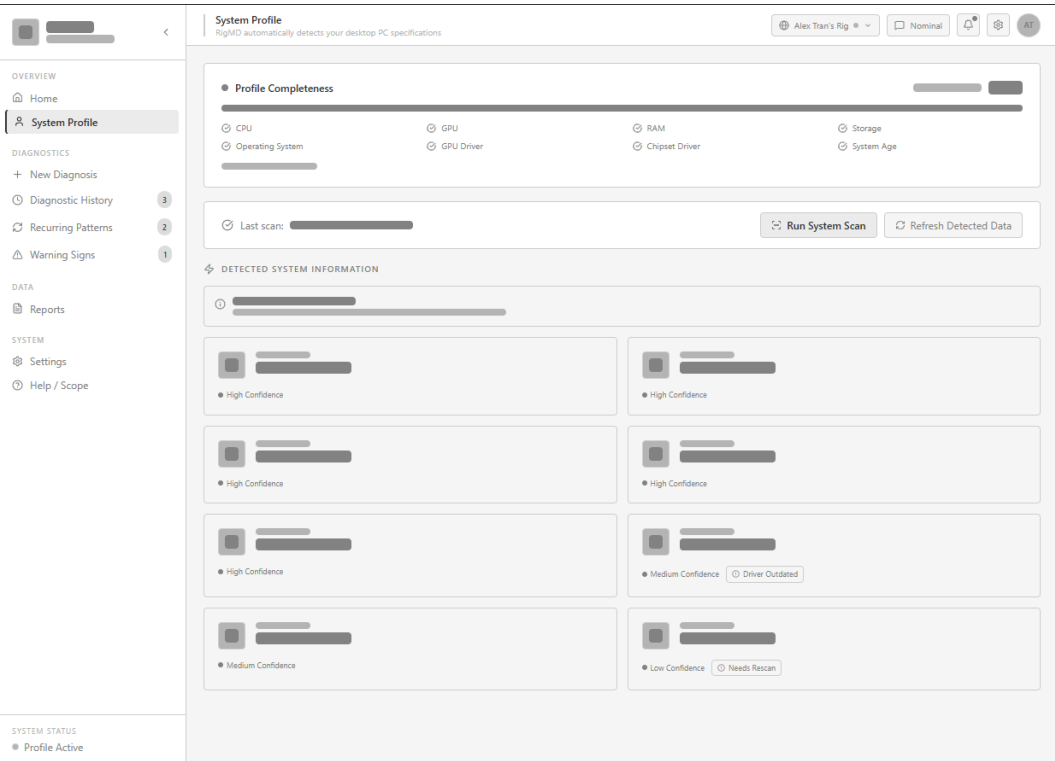
Use Case Diagram



Activity Diagram



Wireframe



Module 2: New Diagnosis and Automatic Scanning Module

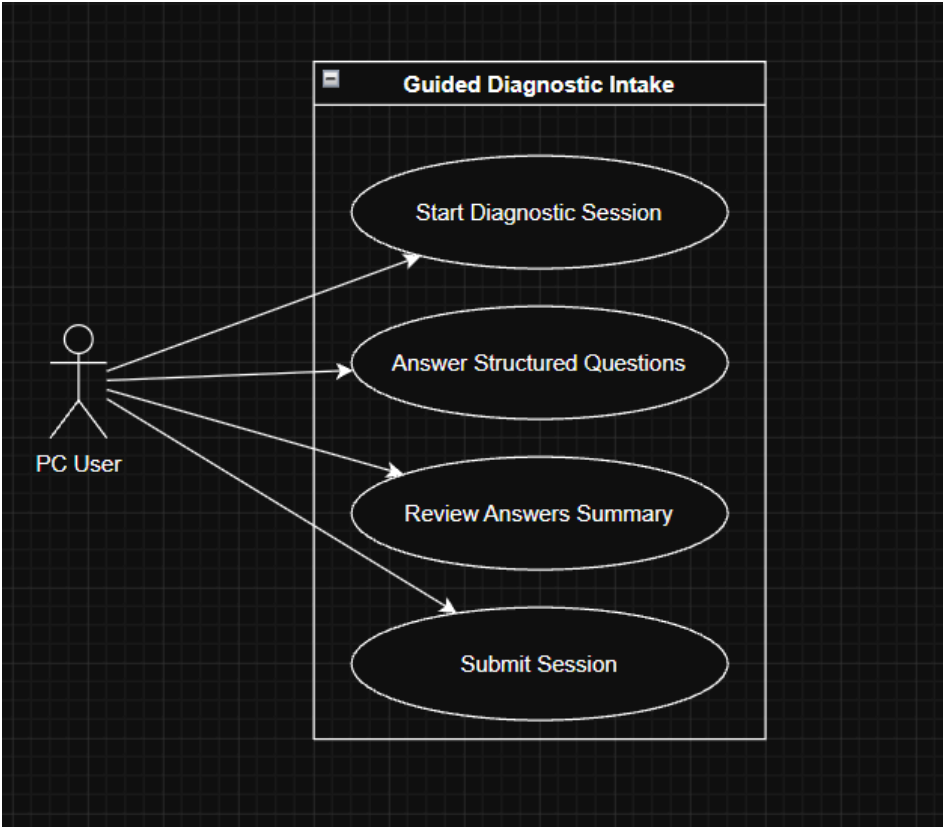
2.1 Guided Diagnostic Intake

Use Case Description

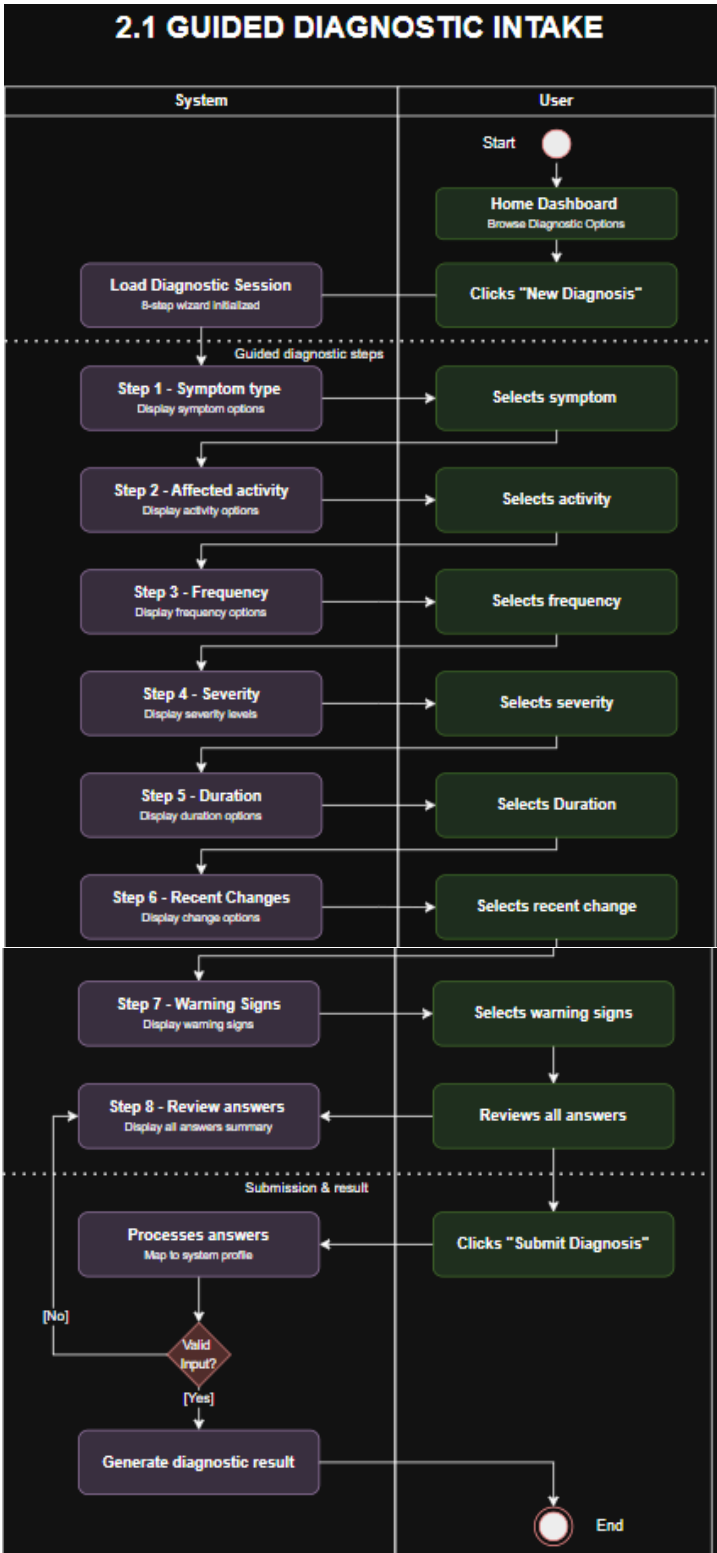
Use Case ID	UC-2.1
Use Case Name	New Diagnosis and Automatic Scan Selection
Actor	PC User

Precondition	The RigMD application and Windows Agent are available.
Main Flow	The user selects New Diagnosis. The system presents the available Full Scan, Component Scan, and Scenario Scan options. The user selects the appropriate scan type and, where applicable, selects the component or scenario to investigate. The system displays a summary of the selected scan. When the user starts the scan, the Windows Agent collects fresh system telemetry and diagnostic evidence relevant to the selected scan. The Agent generates or updates the system snapshot and submits the required evidence to the backend. The Diagnostic Engine evaluates the collected evidence and generates an advisory result.
Alternative Flow	If required telemetry or diagnostic evidence cannot be collected, the system identifies the unavailable information and either continues using sufficient available evidence or informs the user that the selected scan cannot be completed reliably.
Postcondition	A diagnostic session is completed with an evidence-based advisory result or safely terminated with an appropriate status.
Non-functional Requirement	The scan interface shall clearly communicate the current state of the diagnostic process, including collecting, analyzing, completed, and failed states.

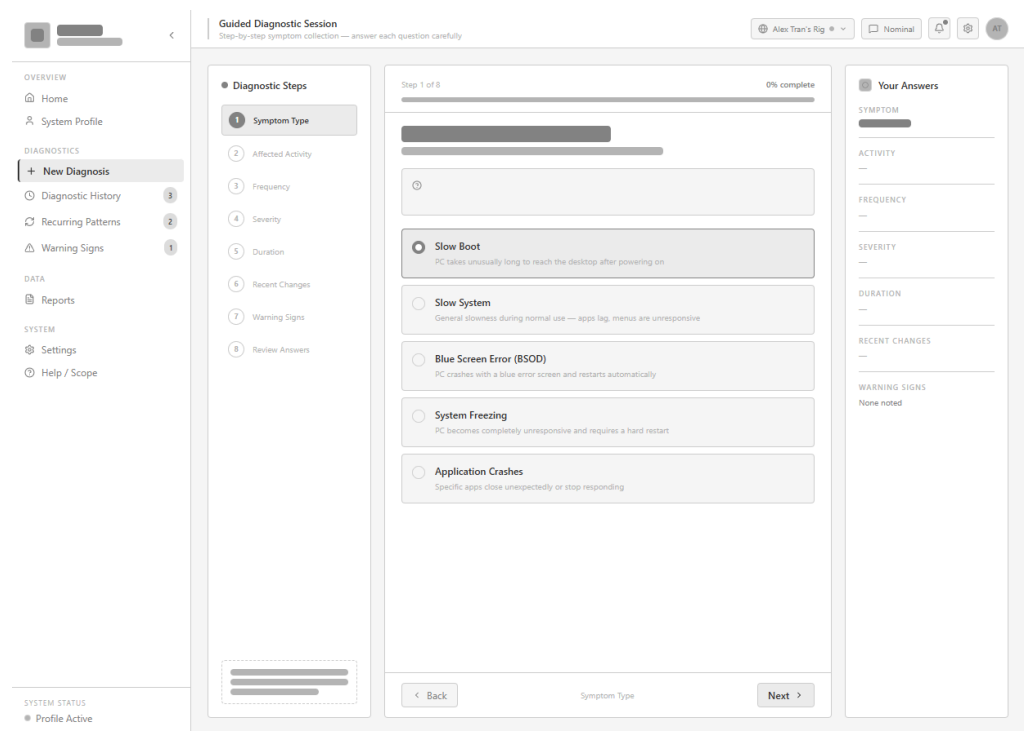
Use Case Diagram



Activity Diagram



Wireframe



Module 3: Evidence-Based Diagnostic Interpretation Engine Module

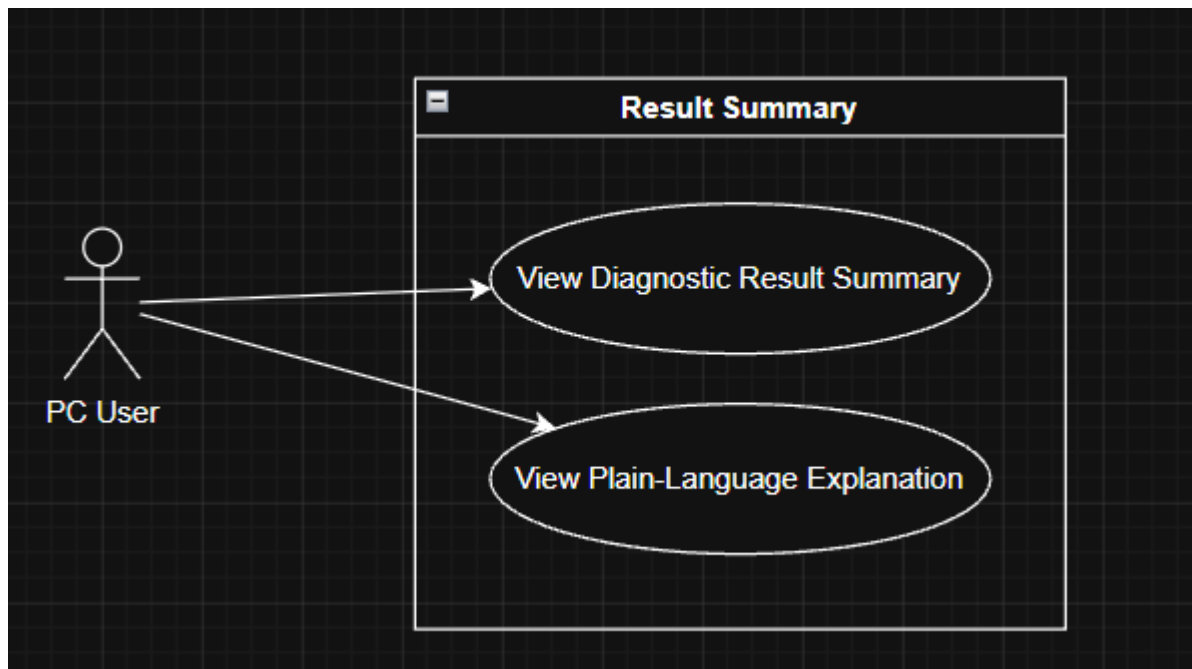
Module 3.1: Result Summary

Use Case Description

Use Case ID	UC-3.1
Use Case Name	Diagnostic Result Summary Visualization

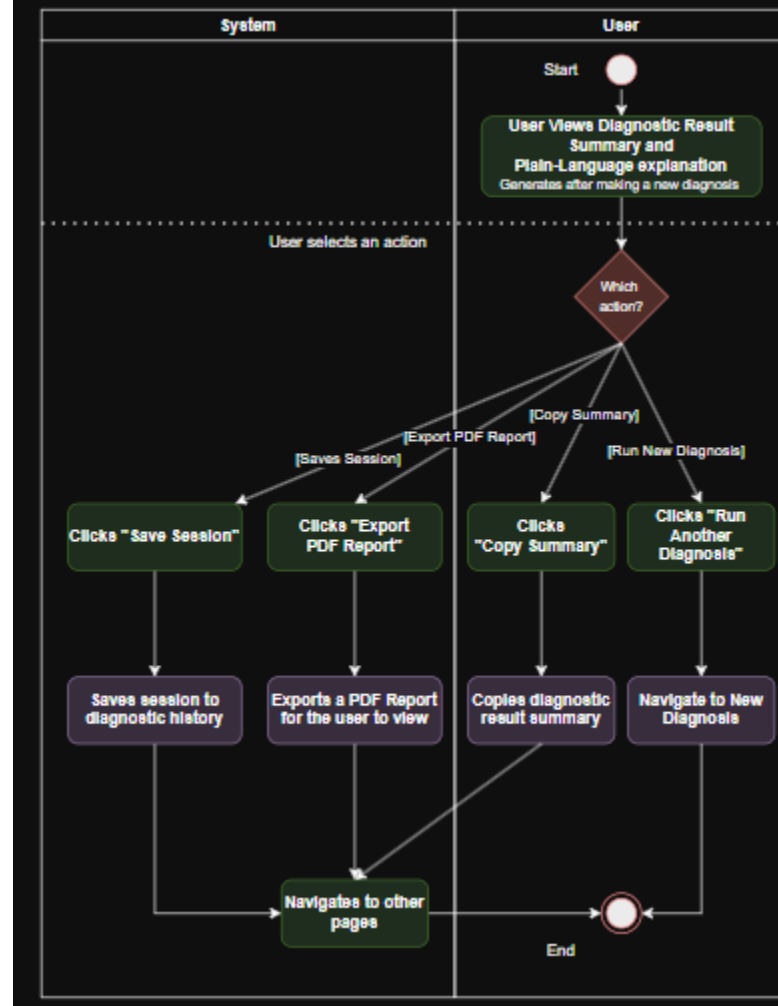
Actor	PC User
Precondition	The system has successfully generated the diagnostic advisory.
Main Flow	The user navigates to the Diagnostic Result screen. The system displays the probable internal cause, diagnostic confidence, and the primary recommended action. The user reads the plain-language explanation to understand why the specific result was provided based on their reported symptoms.
Alternative Flow	If the explanation cannot be generated dynamically, the system displays a default rule-based summary and informs the user.
Postcondition	The user understands the high-level diagnostic finding.
Non-functional Requirement	The result summary must load and render in under 3 seconds.

Use Case Diagram

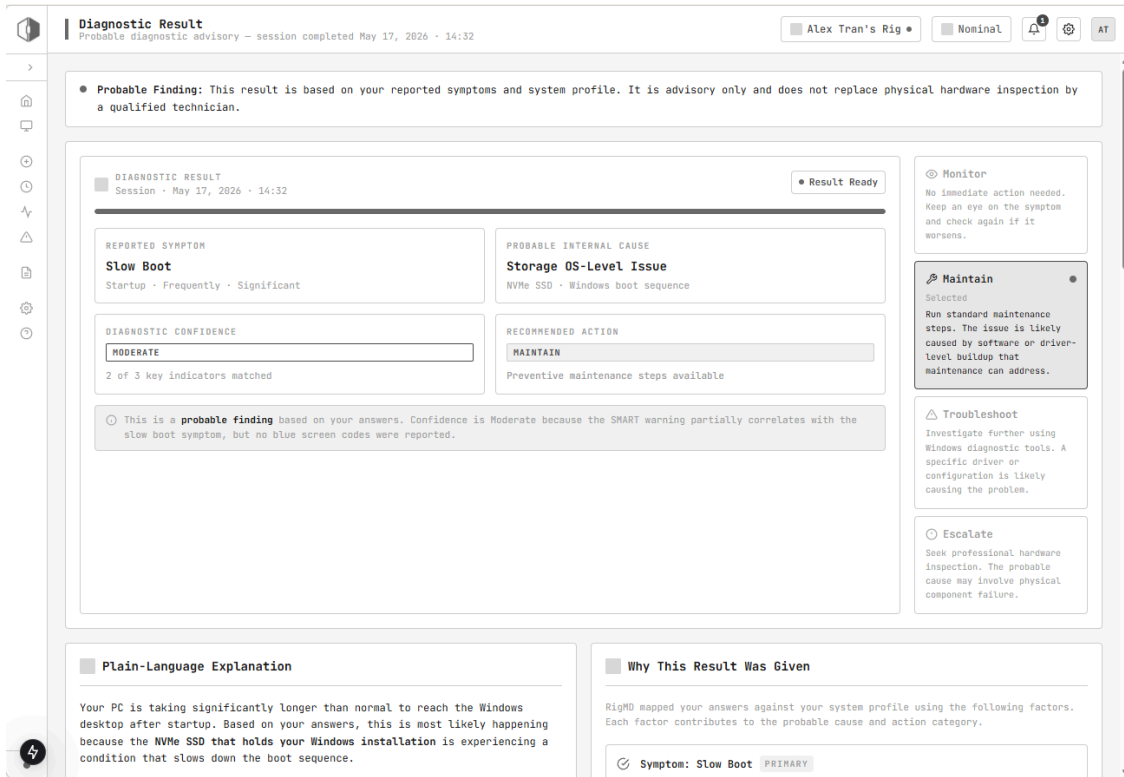


Activity Diagram

3.1 RESULT SUMMARY



Wireframe



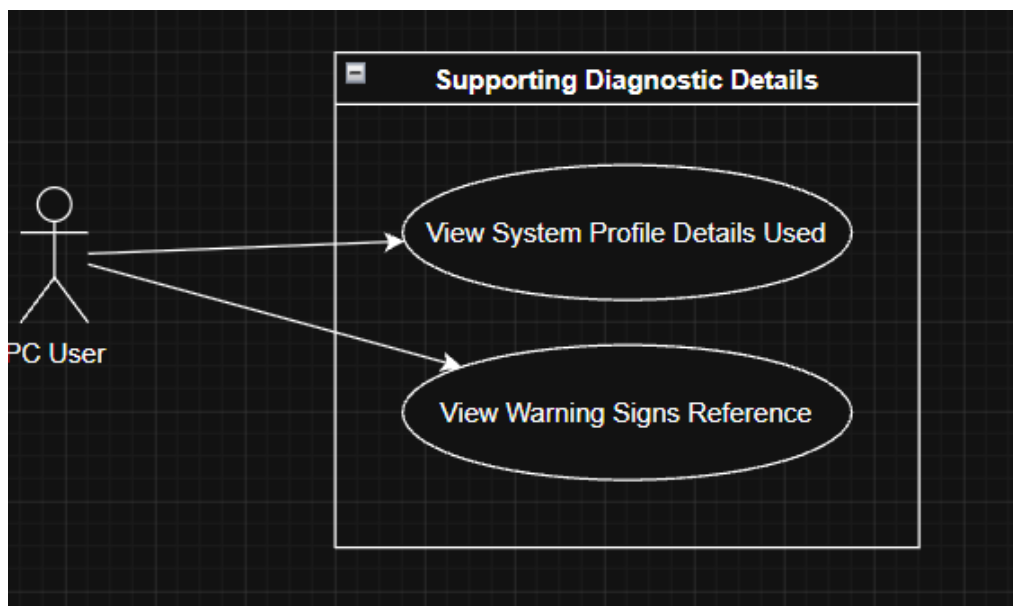
Module 3.2: Supporting Diagnostic Details

Use Case Description

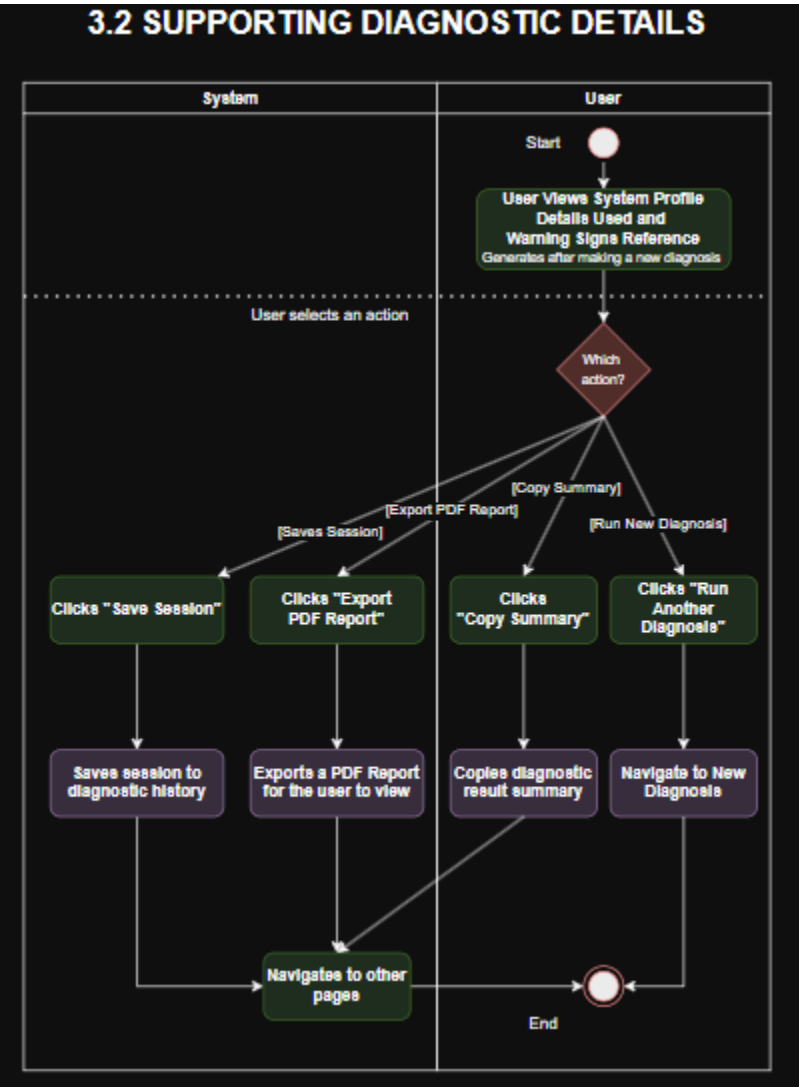
Use Case ID	UC-3.2
Use Case Name	Supporting Diagnostic Details
Actor	PC User
Precondition	The user is viewing a completed Diagnostic Result screen.

Main Flow	The user scrolls down to the Supporting Diagnostic Details section. The system displays the specific system profile fields that directly influenced the result alongside a Warning Signs Reference panel. The user reviews these panels to see exactly which hardware specs and observable indicators were matched during the session.
Alternative Flow	If a profile field was undetected, it is marked as unverified and visually excluded from the reasoning factors.
Postcondition	The user sees the exact data points that triggered the diagnostic result.
Non-functional Requirement	The details panel must accurately reflect the locally stored system profile without modification.

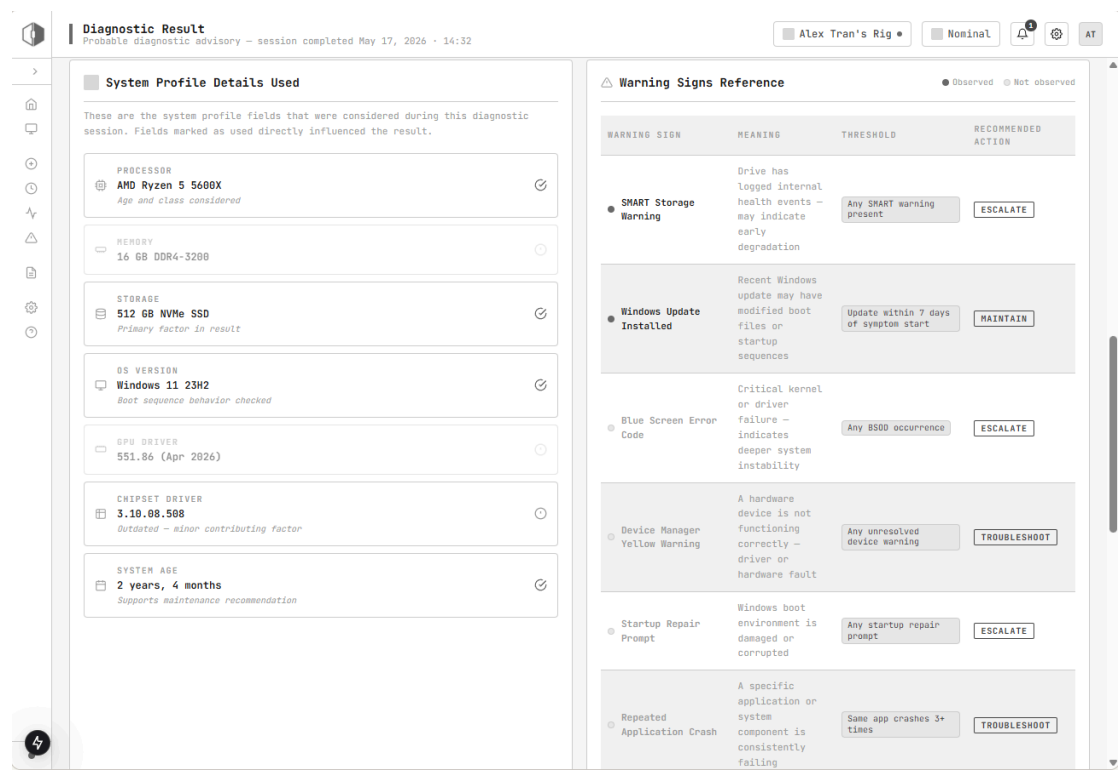
Use Case Diagram



Activity Diagram



Wireframe



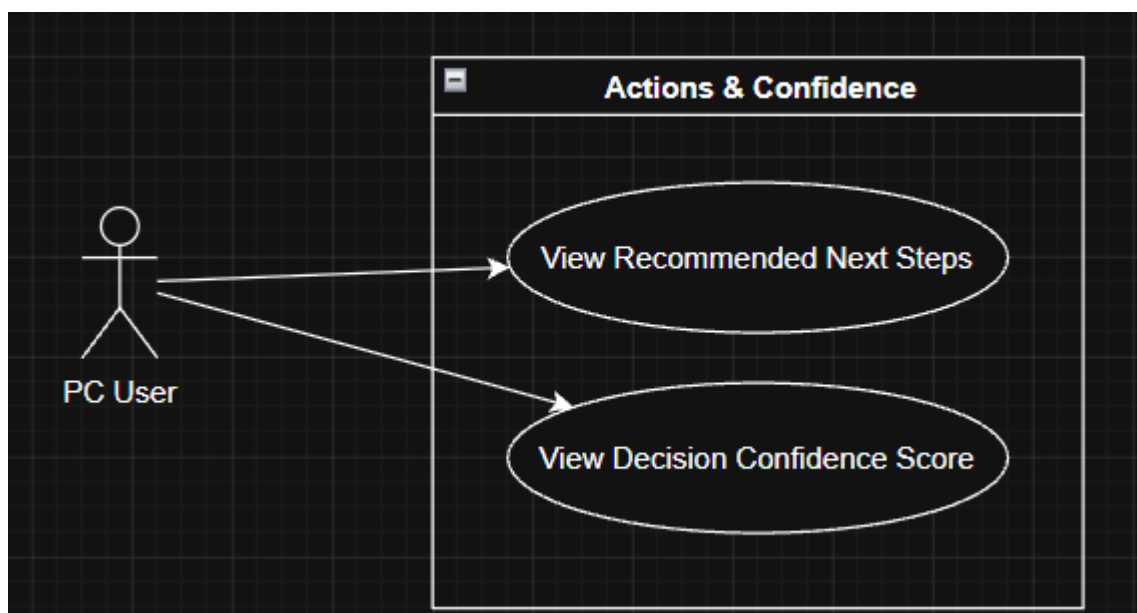
Module 3.3: Recommended Actions and Decision Confidence

Use Case Description

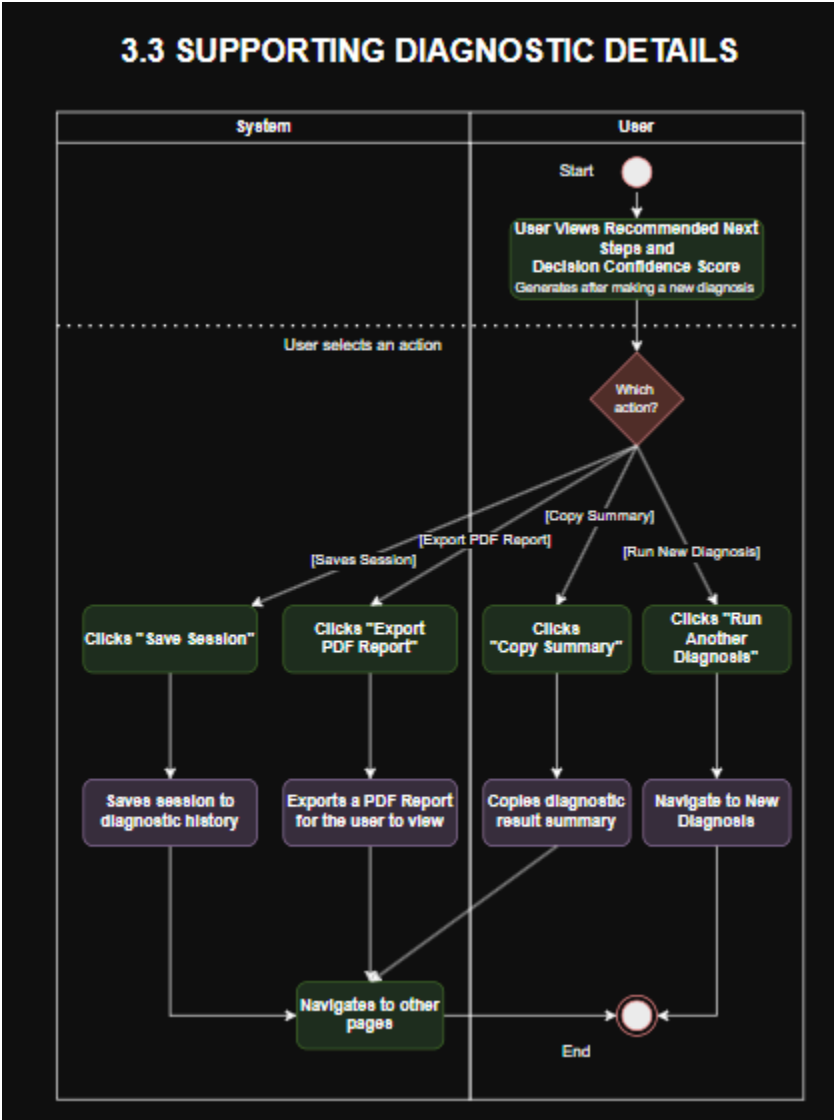
Use Case ID	UC-3.3
Use Case Name	Recommended Actions and Decision Confidence
Actor	PC User
Precondition	The user is viewing the lower section of a completed Diagnostic Result screen.

Main Flow	The user reviews the Recommended Next Steps section. The system displays a prioritized sequence of actions to address the PC issue, complete with priority tags like "DO FIRST". The user then views the Decision Confidence panel to review how much their understanding of the problem has improved since starting the session.
Alternative Flow	If the system classifies the issue strictly as "Monitor," the Next Steps section displays observation instructions instead of active repair tasks.
Postcondition	The user is aware of the necessary steps to resolve or mitigate the issue.
Non-functional Requirement	Recommendations must be written in non-technical plain language.

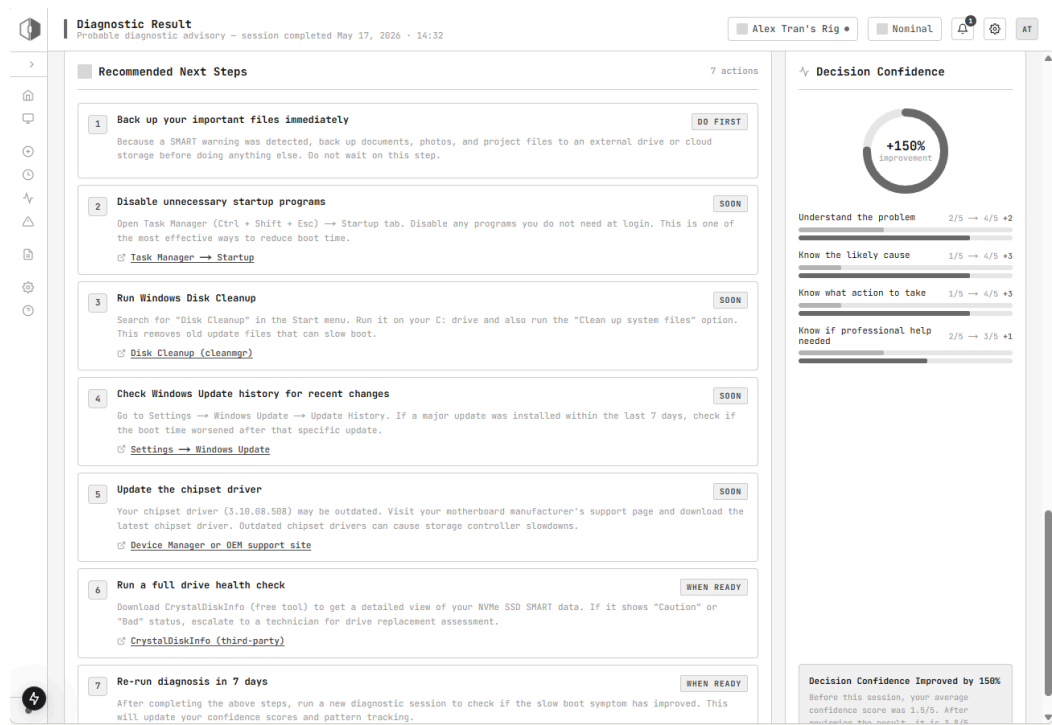
Use Case Diagram



Activity Diagram



Wireframe



Module 4: Advisory Action Module

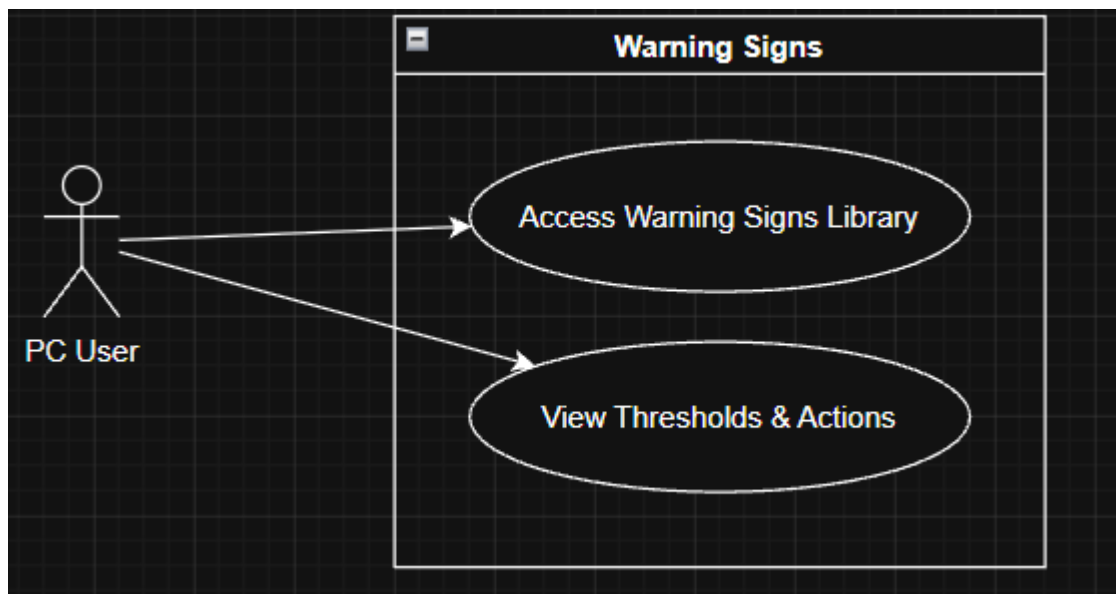
Module 4.1: Warning Signs

Use Case Description

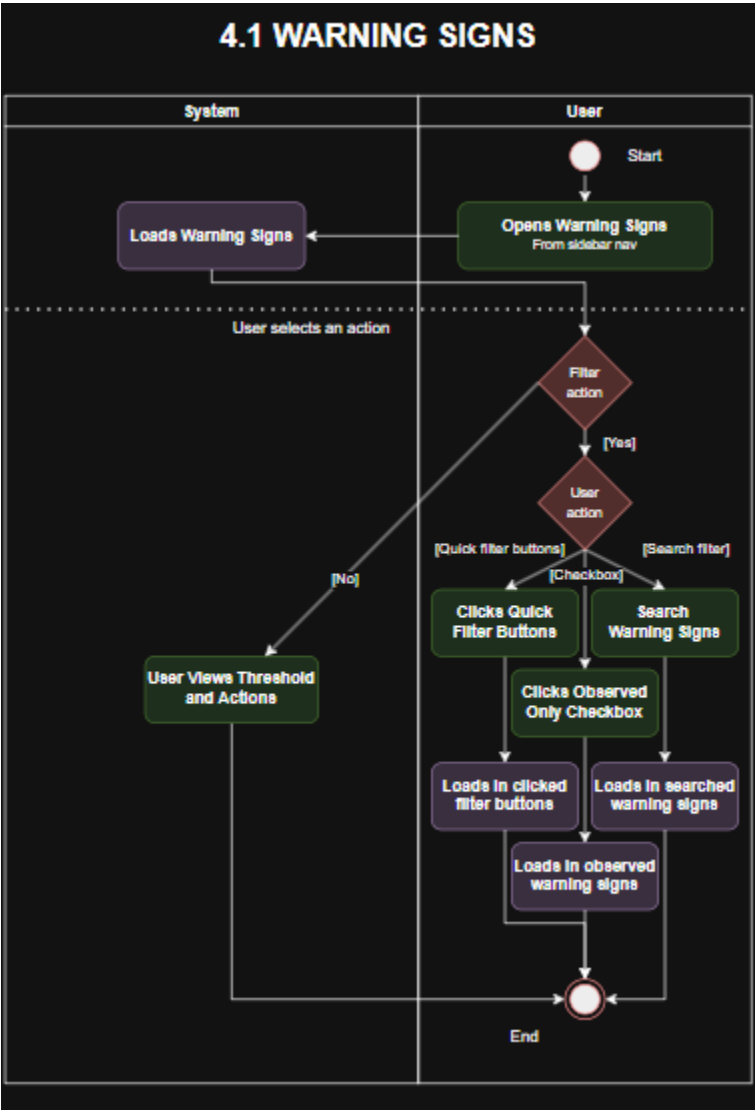
Use Case ID	UC-4.1
Use Case Name	Warning Signs Reference
Actor	PC User
Precondition	The user accesses the main navigation menu.

Main Flow	The user selects the Warning Signs section from the sidebar. The system displays a standalone reference library of observable PC warning indicators. The user reviews the meaning, critical thresholds, and recommended actions for various hardware and software symptoms without needing to complete a full diagnostic session.
Alternative Flow	The user can filter the library by specific categories such as System Crash or Storage using the provided category pills.
Postcondition	The user gains knowledge about system health indicators and escalation points.
Non-functional Requirement	The warning signs table must render completely within 2 seconds.

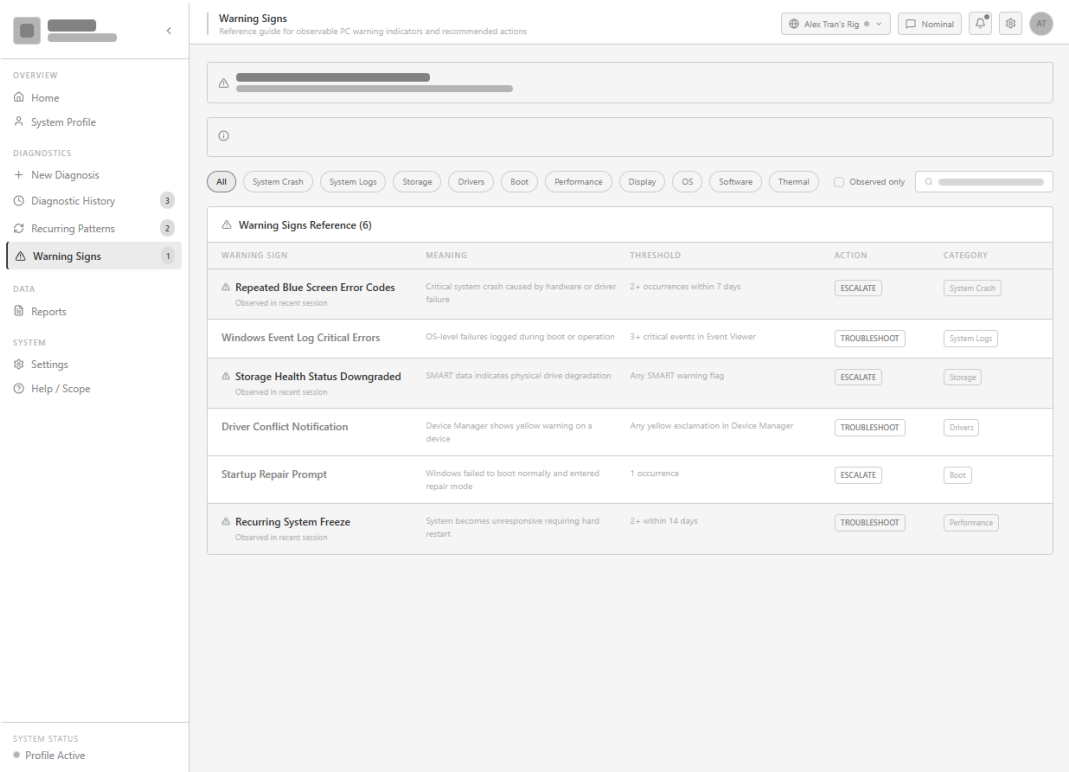
Use Case Diagram



Activity Diagram



Wireframe



Module 5: Transparent Result Dashboard and Report Output Module

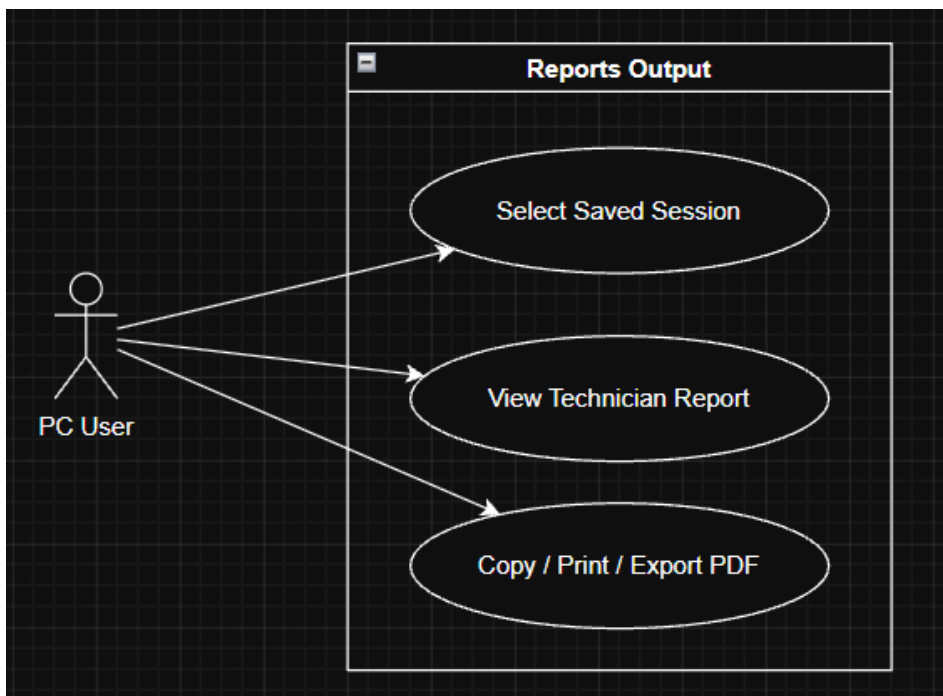
Module 5.1: Reports

Use Case Description

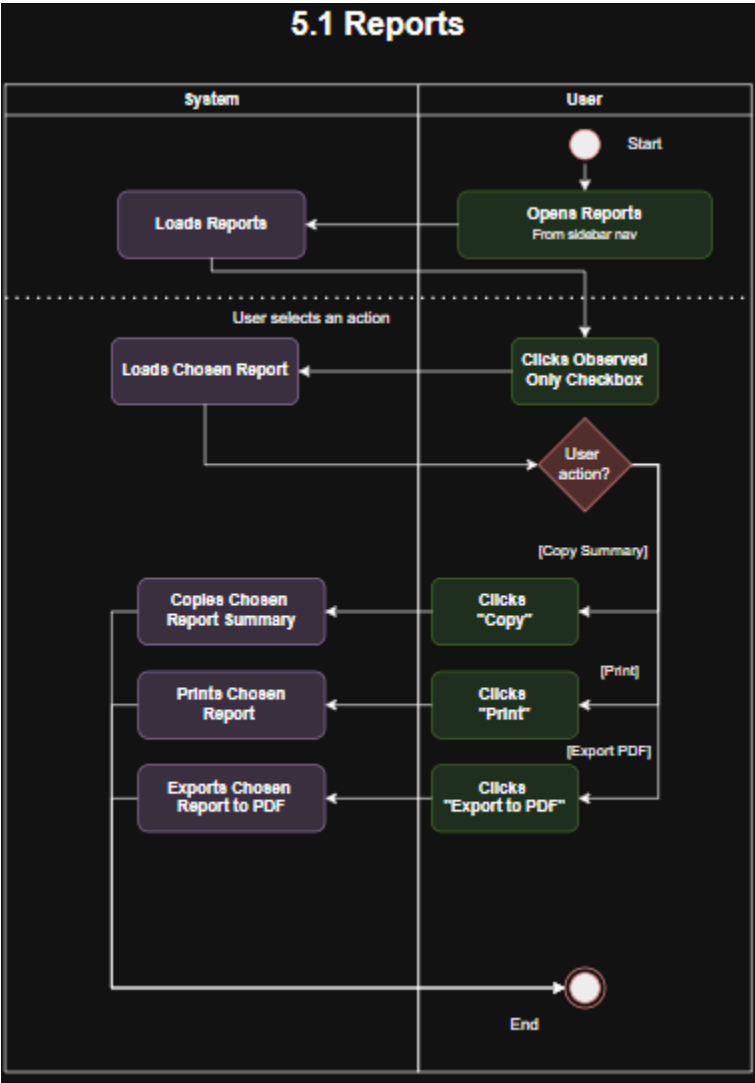
Use Case ID	UC-5.1
Use Case Name	Report Output Generation
Actor	PC User
Precondition	The user navigates to the Reports section.

Main Flow	The user selects a previously completed diagnostic session from the dropdown menu. The system compiles and displays a technician-ready report containing the session summary, profile snapshot, and symptom answers. The user utilizes the provided interface buttons to copy the data, send it to a printer, or export it as a PDF document.
Alternative Flow	If no sessions are saved locally, the system disables the export options and prompts the user to run their first diagnosis.
Postcondition	A shareable report is successfully generated and exported.
Non-functional Requirement	The PDF export must be generated and ready to save within 5 seconds.

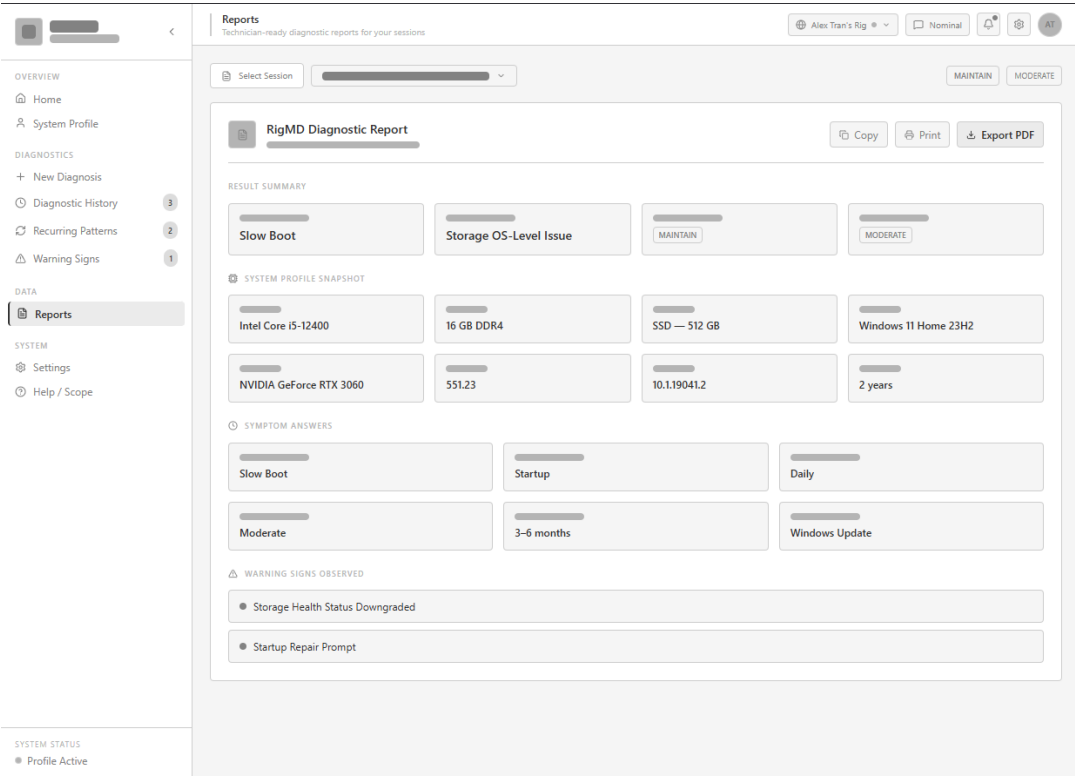
Use Case Diagram



Activity Diagram



Wireframe



Module 6: Session History and Recurring Pattern Detection Module

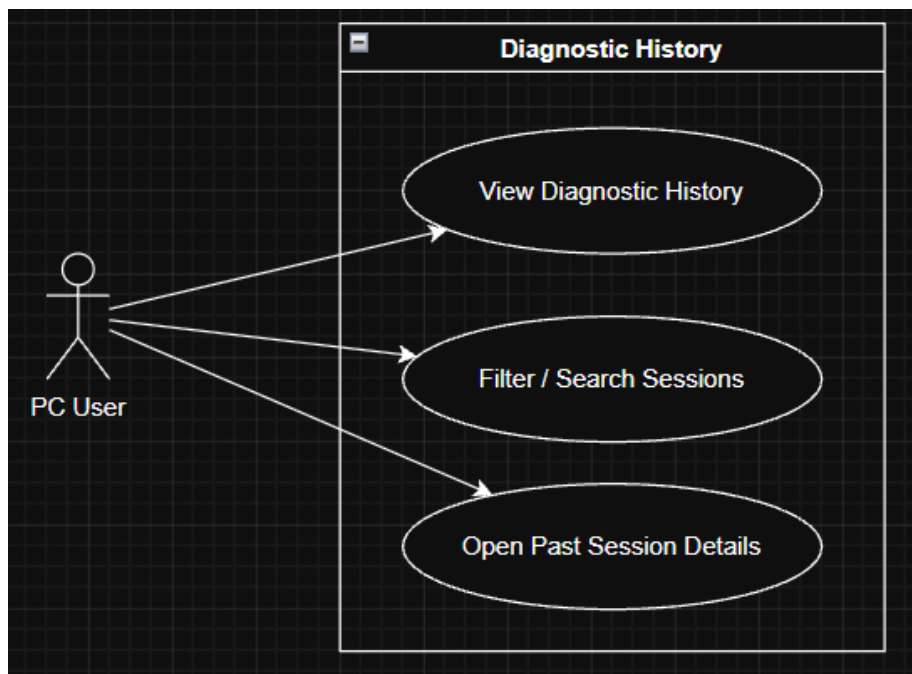
Module 6.1: Diagnostic History

Use Case Description

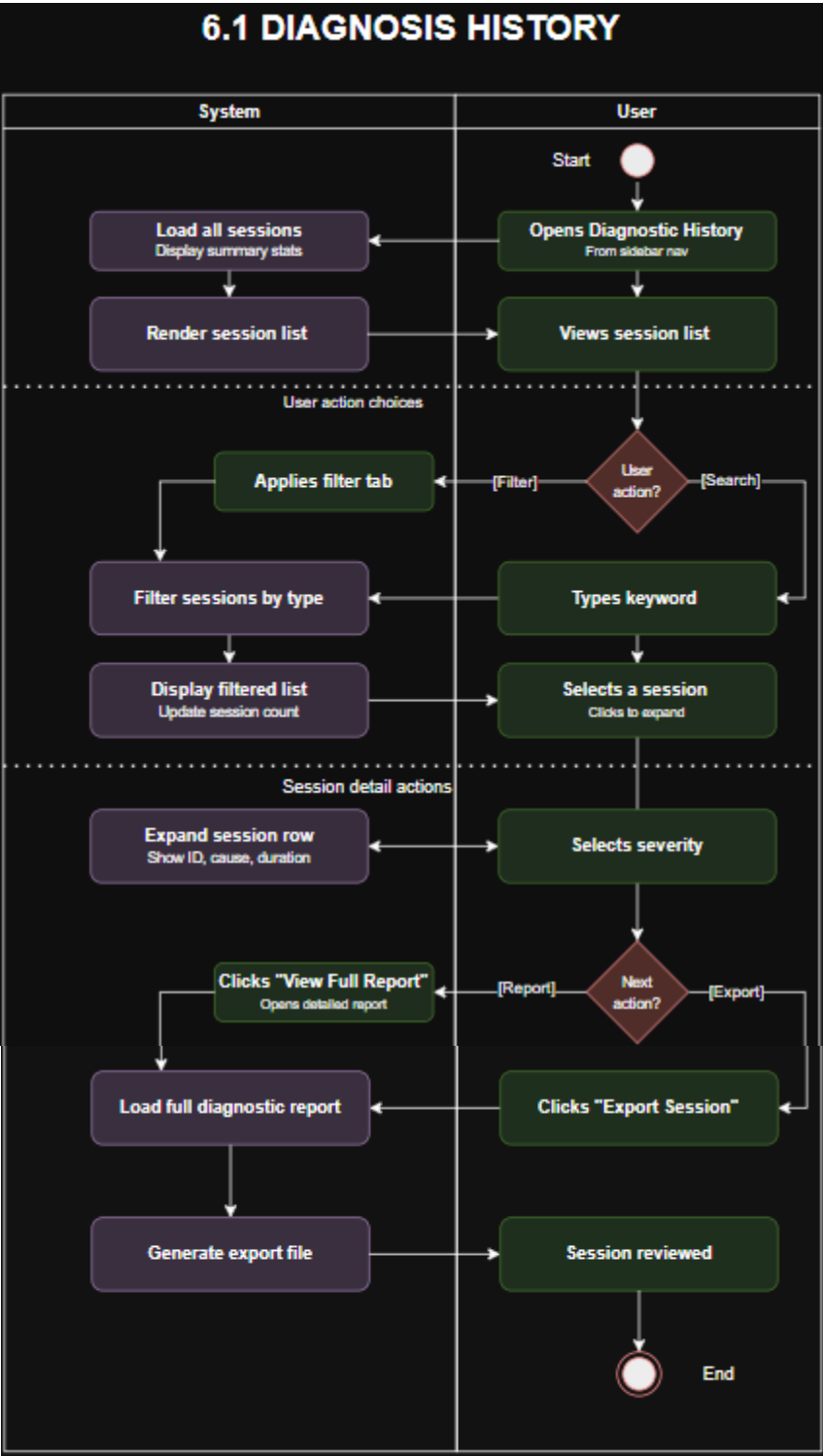
Use Case ID	UC-6.1
Use Case Name	Diagnostic History Management
Actor	PC User
Precondition	The user has completed at least one diagnostic session.

Main Flow	The user navigates to the Diagnostic History section. The system displays a chronological list of all saved diagnostic sessions and their specific action outcomes. The user applies category filters or searches for specific issues. The user clicks on a past session to open and view its specific details and confidence scores.
Alternative Flow	If the local history database is empty, the system displays a placeholder prompt instructing the user to start a new diagnosis.
Postcondition	The user successfully retrieves and reviews their historical PC health data.
Non-functional Requirement	The history list must load up to 50 local records in under 3 seconds.

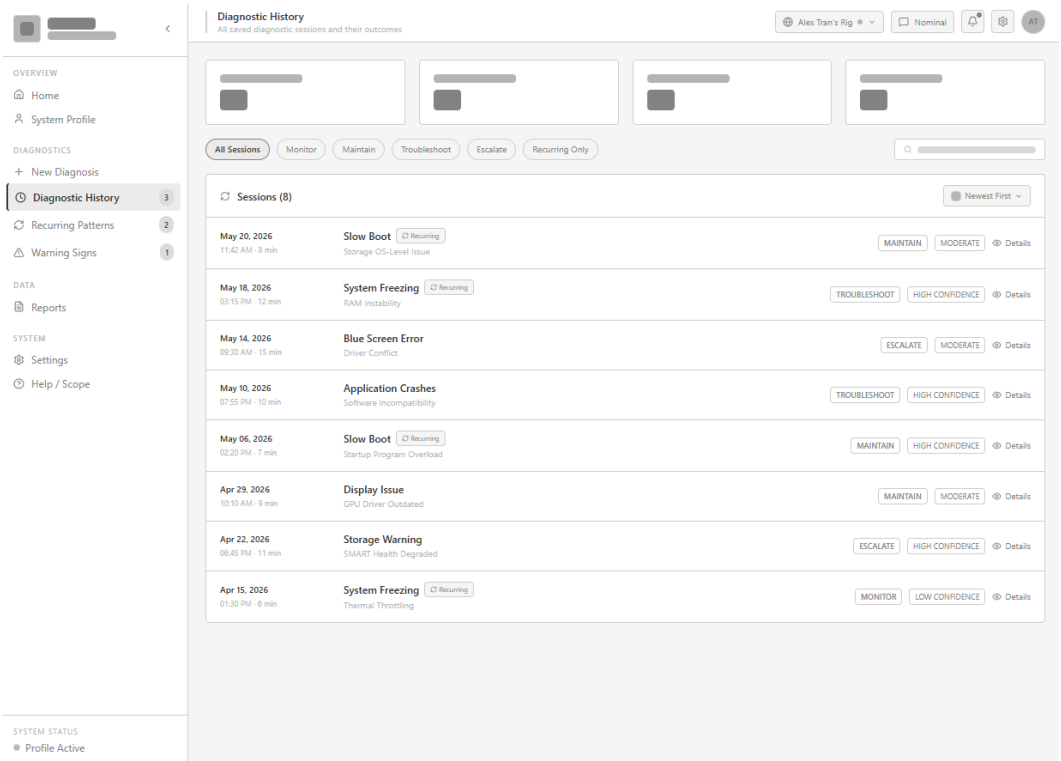
Use Case Diagram



Activity Diagram



Wireframe



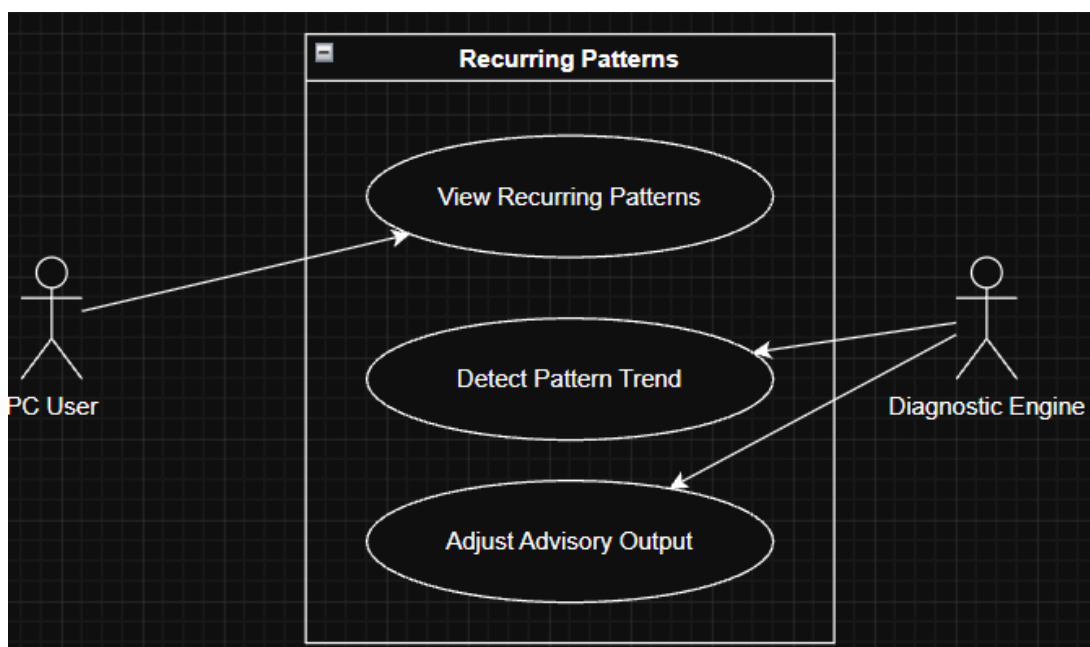
Module 6.2: Recurring Patterns

Use Case Description

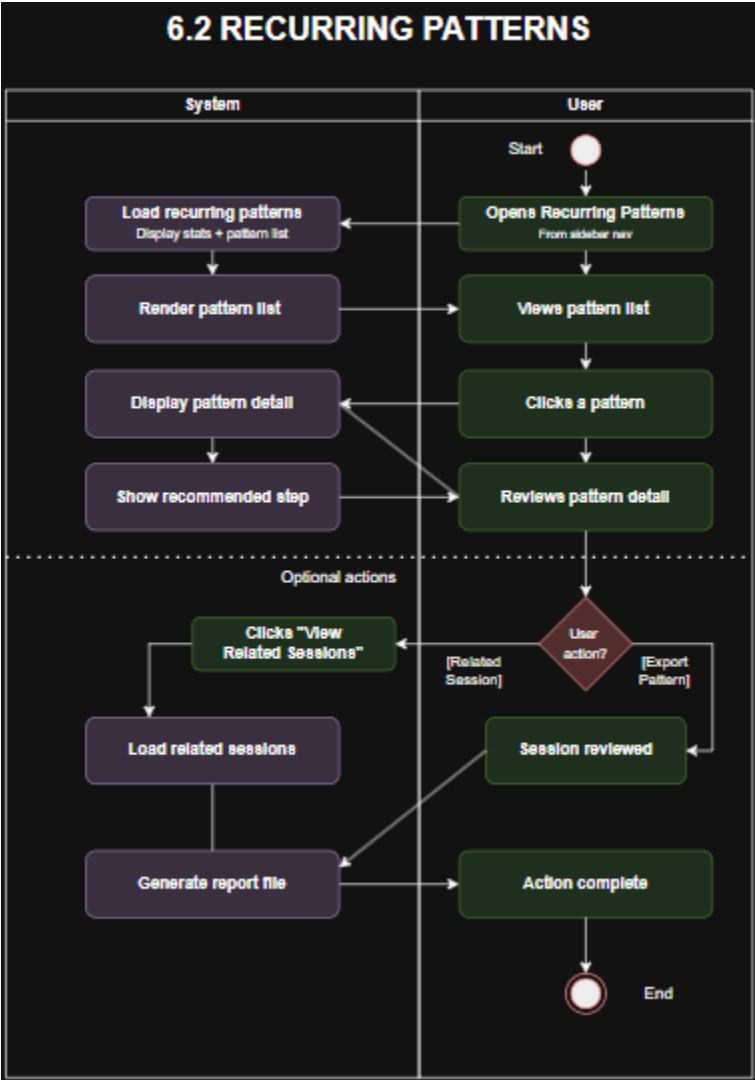
Use Case ID	UC-6.2
Use Case Name	Recurring Pattern Detection
Actor	PC User, Diagnostic Engine
Precondition	The user has completed multiple diagnostic sessions over time.

Main Flow	The user navigates to the Recurring Patterns section. The system retrieves and displays comparisons of repeated symptoms and cause categories across past sessions. The user checks the occurrence count, first seen date, and trend label to determine if the issue is stable, improving, or worsening. The Diagnostic Engine automatically tracks these trends in the background to appropriately adjust future advisories.
Alternative Flow	If no patterns are detected, the system displays a message indicating that the PC currently shows no repeated issues.
Postcondition	The user understands long-term health trends affecting their PC, and the system is primed to escalate worsening conditions.
Non-functional Requirement	Pattern detection must process local history data securely without requiring cloud computation.

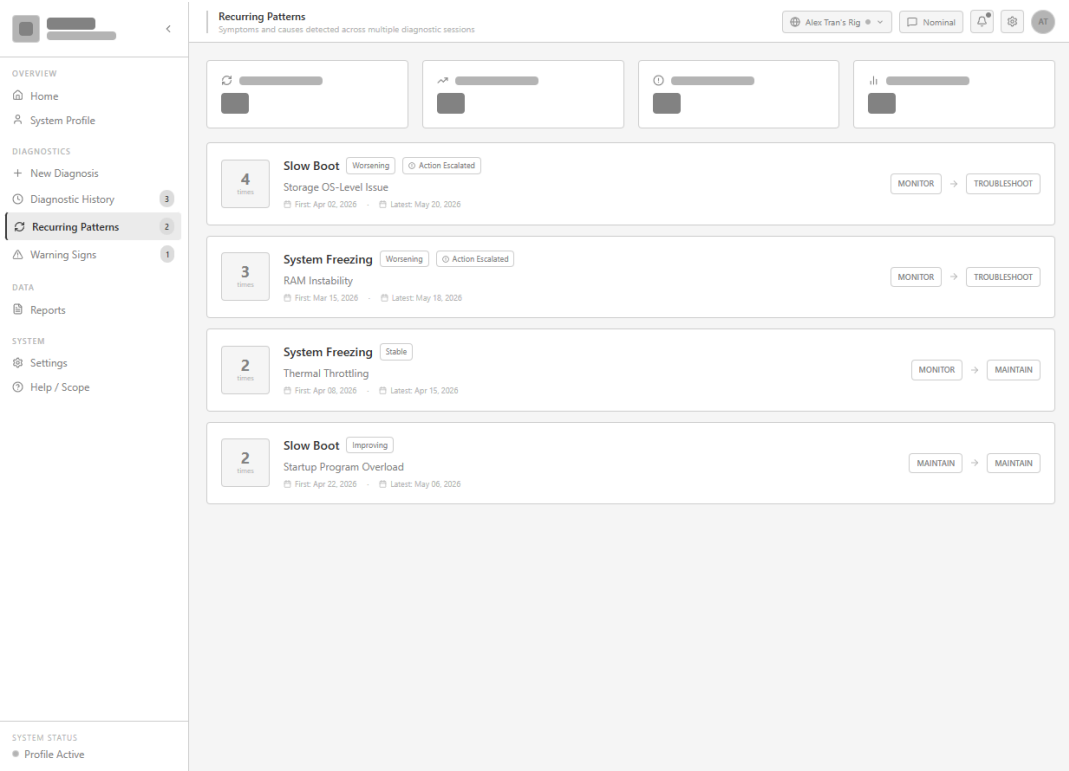
Use Case Diagram



Activity Diagram



Wireframe



3.3 Data Requirements

The following local data must be stored or processed by the application:

Data Item	Required Content
System Profile Record	CPU model, RAM capacity, storage type and capacity, operating system version, GPU driver version, chipset driver version, system age, detection confidence, and last scan timestamp.
Diagnostic Session Record	Session ID, timestamp, selected symptom type, affected activity, frequency, severity, duration, recent changes, system state, warning signs, and reviewed answers.
Diagnostic Output Record	Probable internal cause category, advisory action category, confidence label, reasoning factors, explanation, recommended next steps, and warning signs reference.
Recurring Pattern Record	Repeated symptom type or internal cause category, occurrence count, first seen date,

	latest date, trend label, previous action category, and current action category.
Report Record	Selected session summary, system profile snapshot, symptom answers, warning signs observed, result summary, and export metadata.

Diagnostic information shall be stored and exchanged according to the current RigMD client-server and Windows Agent architecture. Information required for backend coordination, Agent communication, system snapshots, diagnostic workflows, and controlled remediation may be transmitted through the configured backend and Supabase services. The system shall minimize the collection, transmission, and retention of information to data necessary for supported functions. The system shall not require that all diagnostic data be stored exclusively on the user's local device.

3.4 Non-Functional Requirements

Performance

Requirement	Description
Startup Time	The application shall launch and display the main dashboard within 5 seconds on a device meeting the minimum requirements.
Scan Interface Responsiveness	Scan selection, progress updates, and supported user-interface interactions shall respond within an acceptable interactive time under normal operating conditions.
Rule-Based Result Time	The application shall display the diagnostic result within 3 seconds when using offline or rule-based-only mode.
Diagnostic Result Time	The system shall display a deterministic diagnostic result within 3 seconds after all required diagnostic evidence has been collected

	and submitted for analysis, excluding telemetry collection time. Optional AI-assisted explanation generation may require additional processing time depending on network and service availability.
Session Save Speed	Completed session records shall be saved to local storage within 2 seconds of result generation.
History Load Time	The diagnostic history dashboard shall load and display saved session records within 3 seconds for at least 50 saved sessions.
Report Generation Time	A selected session report shall be prepared for viewing, copying, printing, or PDF export within 5 seconds under normal operation.

Security and Privacy

Requirement	Description
Data Storage	System profile, diagnostic session, system snapshot, evidence, and remediation records shall be stored according to the current application architecture. Data required for backend coordination and Agent communication may be stored or exchanged through the configured backend and Supabase services.
Access Limitation	The system shall restrict access to diagnostic data and Agent operations to the application workflows and permissions required by the prototype. The system shall not provide unrestricted remote access to the user's device or system data.
AI API Data Limitation	AI API requests shall include only structured symptom data, system profile categories or values needed for explanation, and the classified

	output. Personally identifiable information shall not be sent.
Encrypted Communication	All AI API communication shall use HTTPS.
Input Validation	All symptom responses, profile values, and local data records shall be validated before processing. Invalid or incomplete data shall produce clear user-facing messages.
Data Integrity	The system shall detect corrupted or unreadable local data files and prevent the application from entering an inconsistent state.
Agent Action Validation	The Windows Agent shall validate all received remediation requests against a predefined set of supported operations before execution.
Agent Action Validation:	The system shall not permit arbitrary CMD, PowerShell, shell, script, or unrestricted command-string execution through the Windows Agent.
Remediation Consent	Applicable remediation actions shall require explicit user consent before execution

Reliability

Requirement	Description
Deterministic Diagnostic Availability	Core diagnostic interpretation shall remain available through deterministic application logic without depending exclusively on an external AI service. Functions that require communication with the backend, Supabase services, or Windows Agent may require network connectivity.
AI Fault Tolerance	If the optional AI-assisted reasoning or explanation service is unavailable, the system shall continue using deterministic diagnostic logic and fallback explanation mechanisms. The

	system shall inform the user when an AI-assisted function is unavailable if that function affects the presented explanation. .
Data Persistence	System profile and session history shall persist across application restarts under normal operation.
Error Handling	The system shall provide plain-language messages for file I/O errors, API timeout, invalid input, corrupted data, and unavailable profile data.
Safety Boundary	The system shall avoid unsupported claims and shall recommend professional inspection when symptoms exceed software-layer resolution.

Usability

Requirement	Description
Plain-Language Interface	The interface shall present scan options, symptom context, diagnostic findings, evidence summaries, explanations, warnings, remediation information, and recommendations in terms understandable to non-technical users.
Guided Diagnostic Workflow	The diagnostic process shall guide users through scan selection and, where necessary, structured symptom context without requiring users to formulate technical diagnoses or provide unrestricted technical free-text input.
Transparency	The result dashboard shall show the reasoning factors, system profile details used, warning signs, and next steps that support the recommendation.
Consistency	The desktop interface shall use consistent navigation, labels, filters, buttons, and card layouts across all screens.

Evaluation Target	The system shall achieve a mean SUS score of at least 75 in usability testing with at least 10 target users.
Decision Confidence Target	The system shall demonstrate at least 25% improvement in composite user decision confidence ratings from pre-session to post-session evaluation.

Diagnostic Accuracy and Validation

Requirement	Description
Expert Validation Threshold	The diagnostic and recommendation outputs shall achieve at least 80% expert-validated acceptance across at least 20 controlled diagnostic scenarios.
Scenario Coverage	Controlled scenarios shall cover slow boot, freezing, repeated blue screen errors, driver conflict behavior, storage warning indicators, application crashes after updates, no-display behavior after a driver change, and basic no-power triage.
Recommendation Relevance	Expert evaluators shall assess whether the advisory action category is appropriate for the given symptom pattern and system profile.
Explanation Clarity	Expert evaluators shall assess whether the explanation accurately connects symptoms, profile data, probable internal cause, and recommended action.
Remediation Safety	The interface shall present scan options, symptom context, diagnostic findings, evidence summaries, explanations, warnings, remediation

	information, and recommendations in terms understandable to non-technical users.
Remediation Execution Prevention	Evaluators shall assess whether the system accurately reports remediation execution status, including successful execution, rejection, failure, and available verification results.
Arbitrary Execution Prevention	The remediation workflow shall be evaluated to verify that unsupported arbitrary CMD, PowerShell, shell, script, or command-string execution cannot be initiated through the system.

4. Validation and Evaluation Requirements

The system shall be evaluated using expert validation, usability testing, and decision confidence measurement.

Evaluation Area	Requirement
Expert Validation	At least two hardware-knowledgeable evaluators shall review the system output across a minimum of 20 controlled scenarios. Each scenario shall include a sample system profile, Windows-observable symptoms, expected internal cause category, and expected advisory action.
Usability Testing	At least 10 target users shall complete a session with the prototype and answer the System Usability Scale after using the system. The target mean SUS score is at least 75.
Decision Confidence Survey	A structured Likert-scale survey shall be administered before and after each session to measure confidence in understanding the problem, identifying the likely cause, understanding the recommendation, and deciding on the next action. The target improvement is at least 25%.

Qualitative Feedback	Short post-test feedback shall be collected around explanation clarity, trust in recommendation, ease of guided workflow, usefulness of warning signs, and value of session history.
Controlled Remediation Evaluation	Supported remediation scenarios shall be evaluated for safety checks, consent handling, predefined-action enforcement, execution reporting, and result verification.
Evidence-Based Diagnosis Evaluation	Expert evaluators shall assess whether the probable cause and advisory action are appropriately supported by the system profile, fresh Agent telemetry, diagnostic evidence, and relevant symptom context.